

الجمهورية العربية السورية
جامعة حمص
كلية الهندسة المعلوماتية
السنة الرابعة - الفصل الدراسي الثاني
مقرر نظم التشغيل (2)

المحاضرة الأولى - الاتصالات والتزامن بين العمليات *Process Communication and Synchronization*



إعداد:

د. علي الحاتم

Dr. Ali Alhatem

الاتصال بين العمليات (Inter-process Communication)

- تقسم العمليات الموجودة في نظام حاسوبي إلى عمليات مستقلة (Independent Processes) و عمليات متعاونة أو متفاعلة (Cooperating Processes).
- العمليات المتعاونة أو المتفاعلة يمكن أن تؤثر أو أن تتأثر بالعمليات الأخرى نتيجة تشاركها معها بالبيانات الموجودة في الذاكرة.
- تحتاج العمليات للتفاعل فيما بينها لتحقيق:
 - مشاركة البيانات فيما بين هذه العمليات.
 - زيادة سرعة انجاز المهام.
 - تجزئة الأعمال التي تقوم البرامج بتنفيذها (Modularity).
 - توفير راحة و فعالية أكبر في التعامل مع النظام الحاسوبي.

مفهوم التوازي (Concurrency) :

نقصد بالتوازي تنفيذ أكثر من عملية في وقت واحد. قد تكون هذه العمليات المتوازية مستقلة عن بعضها البعض (independent processes) أو متعاونة مع بعضها البعض (cooperative processes). مثلاً قد أقوم بتشغيل محرر النصوص (WinWord) لأكتب عليه، واستمع إلى مادة صوتية بواسطة مشغل الوسائط (media player)، ويكون المتصفح يعمل في تنزيل ملفات من الانترنت، هذه العمليات تنفذ بالتوازي في وقت واحد ولكنها مستقلة عن بعضها البعض.

التوازي قد يطبق على مستوى العمليات (concurrent processes) أو على مستوى الخيوط (concurrent threads).

العملية المستقلة هي التي لا تؤثر ولا تتأثر بالعمليات التي تعمل معها في نفس الوقت.
العملية تكون متعاونة إذا أثرت و/أو تأثرت بالعمليات الأخرى التي تعمل معها بالتوازي.

تعاون العمليات:

إذا احتاجت العمليات المتوازية أن تتشارك في بعض الموارد أو أن تؤدي عملاً مشتركاً، فلا بد لها من أن تتعاون مع بعضها البعض. هذا التعاون يتطلب اتصال بين العمليات (أوالخيوط) وعملية تزامن (synchronization).

تعاون العمليات:

الاتصال بين العمليات يتم عن طريق متغيرات مشتركة (shared variables) أو بتبادل الرسائل (message passing). والتزامن نحتاجه ليتم الاتصال في الوقت المناسب، ويتحقق بوضع نقاط تزامن للعمليات بحيث إذا سبقت عملية بقية العمليات ووصلت نقطة تزامن، عليها أن تنتظر بقية العمليات الأخرى لتصل هذه النقطة، وإلا سيكون لدينا نتائج غير متوقعة.

العمليات المتزامنة غالباً ما تتنافس على استخدام الموارد المشتركة بينها بصورة احتكارية (mutually exclusive)، بحيث لا يمكن لعمليتين استخدام نفس المورد في نفس الوقت. أي أنه في أي لحظة يسمح فقط لعملية واحدة باستخدام المورد المشترك.

لماذا تحتاج العمليات المتوازية التعاون فيما بينها:

- طلب عملية لخدمة من عملية أخرى: في هذه الحالة على العملية التي طلبت الخدمة انتظار العملية التي تُطلب منها الخدمة حتى تفرغ منها. مثلاً إذا طلبت عملية من نظام التشغيل معلومة معينة من القرص الصلب، ستظل هذه العملية في انتظار المعلومة حتى تصلها من القرص إلى الذاكرة، أو إذا طلب المتصفح من خادم ويب بالانترنت موقع معين فعلى المتصفح الانتظار حتى يرسل الخادم الموقع المطلوب، فيقوم المتصفح وقتها بعرضه على المستخدم.

لماذا تحتاج العمليات المتوازية التعاون فيما بينها:

- زيادة سرعة التنفيذ: وذلك بتقسيم مهمة واحدة كبيرة إلى مهام صغيرة (عمليات صغيرة) ويتم تنفيذها بالتوازي. قد تسبق عملية في تنفيذها عمليات أخرى معها، فإذا تركنا هذه العملية لتكمل تنفيذها دون انتظار العمليات الأخرى فقد يتسبب هذا في نتائج غير متوقعة أو غير صحيحة، لذلك لا بد من وضع نقاط تزامن لاتتعداها العمليات السريعة ما لم تصل بقية العمليات هذه النقاط.
- قد تعتمد بعض العمليات في عملها على مخرج عمليات أخرى. مثلا إذا استخدمنا الأنابيب الانسيابية (pipeline) بين عمليات فهذا يعني أن مخرج العملية الأولى سيكون مدخل للعملية الثانية، في هذه الحالة لا يمكن للعملية الثانية أن تسبق العملية الأولى. مثلا الأمر التالي (في ينكس):

$\$ ls \mid wc$

هو أمر لتنفيذ عمليتين هما ls ، التي تعرض محتويات الدليل الحالي، و wc لعد الكلمات. هنا لا بد من تزامن (ستنتظر العملية الثانية مخرجات العملية الأولى لتعمل عليها (تحسب عدد كلمات المخرج)).

لماذا تحتاج العمليات المتوازية التعاون فيما بينها:

- قد يكون من الملائم للعمليات أن تعمل مع بعضها لانجاز عمل مشترك، مثلا في نظام الوسائط المتعددة يمكن أن تقوم عملية بإحضار أجزاء المادة الوسائطية من القرص ووضعها في ذاكرة مؤقتة (خازن (buffer))، بينما عملية أخرى تأخذ المادة من الخازن وترسلها إلى جهاز الذي يصدر الصوت/الصورة. هنا تعتبر العملية الأول منتج (producer) بينما العملية الثانية مستهلك (consumer). وعلى المنتج العمل على جعل الخازن ممتلئا دوما بحيث لا يتوقف المستهلك عن العمل (لا يجد الخازن فارغا)، لأن المستهلك إذا وجد الخازن فارغا سيتوقف عن العمل وينتظر وصول بيانات للخازن مما يتسبب في تقطع الفيديو/الصوت أو تشويشه.

أيا كان نوع التعاون بين العمليات، فيجب أن يكون هنالك:

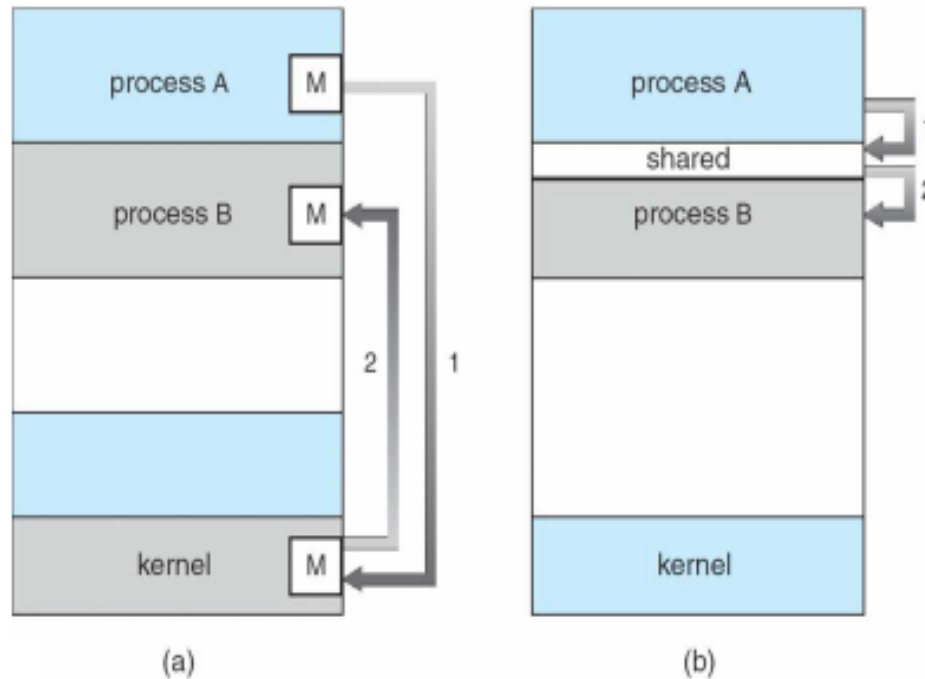
- تزامن بينها.
- اتصال فيما بينها.

الاتصال بين العمليات (Inter-process Communication)

- لكي تتفاعل العمليات فيما بينها يجب على نظام التشغيل توفير إمكانية للاتصال بين العمليات (IPC - Inter Process Communication) الأمر الذي يمكن تحقيقه بإحدى طريقتين:

a. تبادل الرسائل (Message Passing).

b. الذاكرة المشتركة (Shared Memory).



الاتصال بين العمليات (IPC):

عملية متعاونة (Cooperating):

وهي العملية المتعاونة والتي يمكن أن تؤثر أو تتأثر بتنفيذ عملية أخرى في النظام وهذه العملية تستخدم نوعين من الاتصال IPC:

1. تبادل الرسائل (Message passing):

حيث يتم تبادل الرسائل بين العمليات عن طريق الاتصال مع بعضها دون وجود مكان مشترك لتخزين الرسائل ويكون حجم هذه الرسائل ثابت أو متغير وتمر هذه الرسائل على نواة النظام (Kernel) وتحتاج إلى اتصال بين العمليات.

2. الذاكرة المشتركة (Shared memory):

حيث يوجد بين العمليات مكان ذاكره مشتركة لكل منهم ويتم وضع الملفات المشتركة بداخلها.

مثلاً: العملية أ تنتج بيانات والعملية ب تريد أن تقرأ هذه البيانات من العملية أ فإن الذاكرة المشتركة تقوم بهذه المهمة لتسهيل على العملية ب الوصول للمعلومات التي تريدها من العملية أ فهذه بيانات مشتركة يمكن لجميع العمليات الاستفادة منها عند وضعها في الذاكرة المشتركة.

نظم الذاكرة المشتركة (Shared Memory Systems):

(هي عبارة عن عملية مشاركة للذاكرة الافتراضية بين عمليتين أو أكثر، حتى يتم نقل وتبادل البيانات والاتصال بين كافة العمليات).²⁸

المشاكل التي تواجه المستخدمين والمنتجين بالنسبة للذاكرة المشتركة:

المخزن المؤقت غير المحدود (unbounded-buffer):

يمكن للمستخدم أن يقوم بعمليات غير محدودة فهي غير محدودة الحجم

المخزن المؤقت المحدود (bounded-buffer):

يكون هناك مساحة معينة للعمليات بقدر ما تنتج العملية الأولى من معلومات بقدر ما تستهلك العملية الثانية من هذه المعلومات فهي محدودة الحجم

الرسائل العابرة (Message Passing):

وهي عبارة عن رسائل تتم بين العمليات لتبادل البيانات لكن قبل أن يتم الإرسال يجب أن يتم تحديد المرسل والمستقبل وأيضا يجب أن يكون بين الجهتين حلقة اتصال حتى يتم التبادل.

وهنا ثلاث طرق لتنفيذ وصلة اتصال منطقية وعمليات ارسل() / استقبل():

1. الاتصال المباشر أو غير المباشر.
2. الاتصال المتزامن أو غير المتزامن.
3. المخزن المؤقت (buffer)

الرسائل العابرة (Message Passing):

1. الاتصال المباشر وغير المباشر:

المباشر: في هذا النوع من الاتصال نحتاج إلى معرفة المرسل ومعرفة المستقبل أي من الضروري تسمية الجهات حتى يتم الاتصال بينهم وفي هذا النوع لا نحتاج إلى مخزن مؤقت (buffer) لأن الاتصال يتم مباشرة من المرسل إلى المستقبل.

يتم الاتصال بهذه الطريقة:

send (receiver_process, message)

ارسل() : send()

اسم المستقبل هو : receiver_process

والرسالة المرسل: message

وبنفس الطريقة للمستقبل من حيث نداء النظام system call:

receive (sender_process, message)

استقبل() : receive

اسم المرسل: sender_process

الرسالة المرسل: message

الرسائل العابرة (Message Passing):

غير المباشر: في هذا النوع لا نحتاج إلى اسم المستقبل أو المرسل، بل يتم إرسال الرسالة لصندوق البريد (mailbox)، حيث يتم إرسال الرسائل واستقبالها من صندوق البريد. الطريقة كالتالي:

Send (mailbox, message)

receive (mailbox, message)

وفي هذا الاتصال كل عملية لها صندوق بريد، وعادة يتم وضعه على شكل طابور (queue).
خصائص صندوق البريد:

1. لكل صندوق معرف خاص (unique identification)
2. وكل عملية تتصل مع الأخرى عن طريق صندوق البريد
3. أي عمليتين يتم الاتصال بينهما فقط إذا وجد صندوق بريد مشترك

خصائص وصلة الاتصال (CL) communication link:

1. يتكون الرابط بين عمليتين فقط إذا وجد صندوق بريد مشترك بينهما
2. من الممكن الربط بين أكثر من عمليتين
3. إذا وجد بين عمليتين أكثر من رابط، فإن كل رابط مخصص لصندوق بريد معين
4. الرابط ممكن أن يكون أحادي أو ثنائي الاتجاه

الرسائل العابرة (Message Passing):

2. الاتصال المتزامن وغير المتزامن:

1. الاتصال المتزامن المحظور (blocking):

معنى الاتصال المتزامن المحظور أن الجهة المرسل لا تستمر بإرسال الرسائل إلا عندما تصل الرسالة إلى الجهة المستقبل، أي أنه تنتهي عملية الإرسال بوصول الرسالة إلى المستقبل وهذا الاتصال يعتبر آمن وأكثر بطئاً من غير المتزامن.

2. الاتصال المتزامن غير المحظور (unblocking):

الاتصال غير المتزامن غير المحظور (unblocking) تستمر الجهة المرسلة بالإرسال دون التأكد من وصولها للمستقبل، أي أنها ترسل الرسالة بطريقة موازاة (parallelism) ومن مميزاته السرعة لكنه أكثر خطورة من حيث الأخطاء بالنسبة للمتزامن.

الرسائل العابرة (Message Passing):

3. المخزن المؤقت buffering:

لكل رابط بين العمليات سعة معينة لعدد من الرسائل التي تمر من خلاله. السعة صفر zero : بمعنى أن الطابور (queue) طوله صفر أي لا يوجد أي رسائل به ومن المفترض هنا أن تكون العملية تزامنية. بمعنى أن المرسل يجب أن ينتظر حتى تصل الرسالة إلى المستقبل وتسمى هذا الحالة الملتقى (rendezvous)

السعة محدودة : بمعنى أن الطابور (queue) يكون طوله n . إذا كان الطابور للمستقبل غير ممتلئ، يمكن وضع رسالة جديدة فيه، والمرسل يستمر في إرسال الرسائل. أما إذا كان الطابور ممتلئ المرسل يحظر الإرسال (block send) حتى يوجد مكان متاح للرسالة في الطابور.

السعة غير المحدودة: بمعنى أن الطابور يكون طوله إلى ما لا نهاية، وبهذه الحالة تستمر الجهة المرسل بالارسال دون توقف.

اللقاء (Rendezvous):

نفترض أن شابين قد تواعدا على أن يلتقيا في منتزه لم يرياه من قبل، بالفعل قد حضرا كل واحد على حدا، ولكنهما إندهشا من كبر حجم المنتزه، وبالتالي صعوبة أن يجد أحدهما الآخر. هنا على كل شاب أن يختار واحد من أمرين:

- أن ينتظر في مكان واحد ويتوقع أن الآخر سيبحث عنه.
 - أن يبدأ بالبحث بافتراض أن الآخر سينتظر في مكان واحد.
- السؤال هنا أي استراتيجية يجب أن يختارا حتى يكون احتمال اللقاء أكبر ؟
- إذا قرر الإثنين الانتظار فبالأكيد لن يجد أحدهما الآخر أبدا.
 - إن قررا أن يبحثا عن بعضهما فهناك فرصة في ان يتقابلا.
 - إذا قرر أحدهما أن ينتظر والآخر أن يبحث فحتما سيلتقيا (من ناحية نظرية).

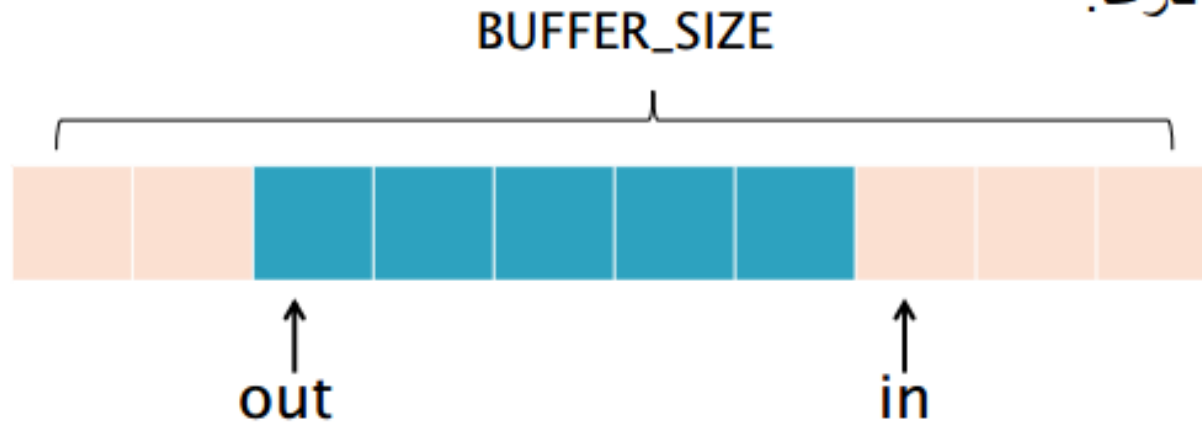
الاختلاف الأساسي بين حواسيب تمرير الرسائل و المعالجات المشتركة بالذاكرة هو تنفيذ عمليات الاتصال على مستوى أنظمة الإدخال/الإخراج بدلاً من إجرائها في ذاكرة النظام.

	الذاكرة المشتركة	تبادل الرسائل
الأداء	جيد وفعال	جيد للتطبيقات التي لا تحتاج إلى اتصال كثيف بين المعالجات
التطبيق	سهلة التطبيق ولكنها مكلفة مادياً	سهلة التطبيق ومنخفضة التكلفة
وسيلة الاتصال	ذاكرة موحدة لجميع العمليات	عبر خطوط الاتصال في الشبكة
السلبيات	يجب توفير أساليب حماية الذاكرة المشتركة وتنظيم الوصول إليها	تكلفة تمرير الرسائل عالية جداً لأنها تتطلب عمليات دخل خرج كثيرة

مسألة المنتج - المستهلك

مسألة المنتج - المستهلك (Producer-Consumer Problem)

- يقوم المنتج بإنتاج وحدات من البيانات يتم استهلاكها من قبل المستهلك.
- تشترك عمليتان بمخزن وسيط (Buffer) ثابت الحجم، بحيث تضع إحداهما (المنتج) المعلومات في المخزن، و تأخذ الأخرى (المستهلك) المعلومات منه.
- تكمن المشكلة في أنه يجب منع المنتج من وضع عنصر جديد في المخزن عندما يكون المخزن ممتلئاً، كما أنه يجب منع المستهلك من قراءة عنصر من المخزن عندما يكون فارغاً.



مسألة المنتج – المستهلك

مسألة المنتج – المستهلك (Producer-Consumer Problem)

البيانات المشتركة – Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Count : عداد لعدد العناصر الموجودة فعلاً في الـ Buffer

مسألة المنتج – المستهلك

مسألة المنتج – المستهلك (Producer-Consumer Problem)

المنتج – Producer

```
while (true) {  
    /* produce an item and put in nextProduced */  
  
    while (count == BUFFER_SIZE) ; // do nothing  
  
    buffer [in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
    count++;  
}
```

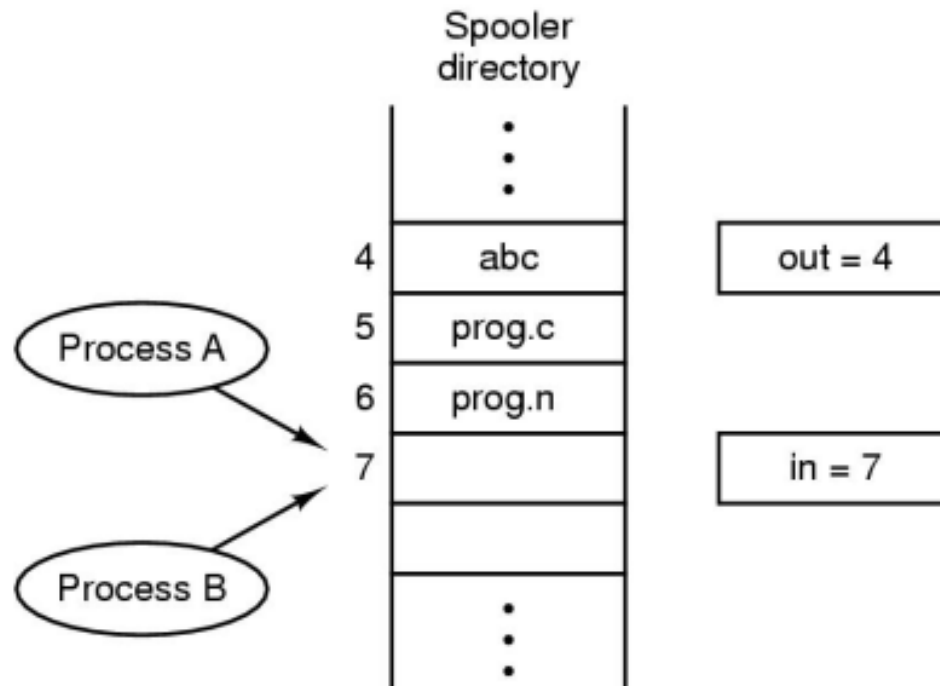
المستهلك – Consumer

```
while (true) {  
    while (count == 0) ; // do nothing  
  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    count--;  
  
    /* consume the item in nextConsumed  
}
```

مسألة المنتج - المستهلك

مسألة المنتج - المستهلك (Producer-Consumer Problem)

حالة السباق - Race Condition



مسألة المنتج – المستهلك

مسألة المنتج – المستهلك (Producer-Consumer Problem)

حالة السباق – Race Condition

- قد تتأثر أو تؤثر العمليات المتعاونة على نتيجة تنفيذها في النظام التي تتشارك فيه منطقة من الذاكرة، حيث أن قدرة كل من هذه العمليات على القراءة و الكتابة إلى هذه الذاكرة المشتركة تؤدي إلى كون النتيجة متعلقة بتسلسل التنفيذ الفعلي (بمن يعمل أولاً)، و هو ما يدعى بحالات السباق (Race Condition).

- التعليمة `count++` تنفذ بالشكل التالي عندما يتم تحويلها إلى لغة الآلة.

```
register1 = count  
register1 = register1 + 1  
count = register1
```

- التعليمة `Count--` تنفذ بالشكل التالي عندما يتم تحويلها إلى لغة الآلة.

```
register2 = count  
register2 = register2 - 1  
count = register2
```

مسألة المنتج - المستهلك

مسألة المنتج - المستهلك (Producer-Consumer Problem)

حالة السباق - Race Condition

- تبعاً لجدولة العمليات فإنه قد ينتج لدينا تسلسل التنفيذ التالي (باعتبار أن قيمة $\text{count}=5$ عند بدء التنفيذ):

S0: producer execute $\text{register1} = \text{count}$ {register1 = 5}
S1: producer execute $\text{register1} = \text{register1} + 1$ {register1 = 6}
S2: consumer execute $\text{register2} = \text{count}$ {register2 = 5}
S3: consumer execute $\text{register2} = \text{register2} - 1$ {register2 = 4}
S4: producer execute $\text{count} = \text{register1}$ {count = 6}
S5: consumer execute $\text{count} = \text{register2}$ {count = 4}

حالات السباق و المناطق الحرجة

• حالة السباق (Race Condition)

هي حالة تقوم فيها عدة عمليات وفي نفس الوقت بالوصول و إجراء عمليات تعديل على بنية معطيات مشتركة و نتيجة لذلك تعتمد النتيجة النهائية الناجمة عن تنفيذ هذه العمليات على التسلسل الذي يتم فيه الوصول لبنية المعطيات المشتركة هذه.

• المناطق الحرجة (Critical Section)

هي جزء من النص البرمجي الخاص بعملية ما، تقوم من خلاله هذه العملية بإجراء عملية تعديل على بنية معطيات مشتركة مع عمليات أخرى في النظام.

• مسألة المناطق الحرجة (Critical Section Problem)

تتمثل هذه المسألة بالحاجة إلى إيجاد طريقة لمنع أكثر من عملية واحدة من قراءة و كتابة البيانات المشتركة في نفس الوقت. بشكل مكافئ نحتاج إلى طريقة للتأكد من أنه إذا كانت عملية ما تقوم بالتنفيذ في منطقة حرجة فإن العمليات الأخرى ستمنع من التنفيذ في مناطقها الحرجة، و هو ما يدعى بالإقصاء التبادلي (Mutual Exclusion)

مسألة المناطق الحرجة

مسألة المناطق الحرجة (Critical Section Problem)

- حل مسألة المناطق الحرجة يتمثل بتصميم بروتوكول تقوم العمليات بالتعامل مع بنى المعطيات المشتركة من خلاله:
 - يجب على كل عملية أن تطلب السماح بالدخول إلى المنطقة الحرجة قبل أن تكون قادرة على تنفيذ التعليمات التي تتعامل مع الذاكرة المشتركة، الجزء من النص البرمجي الذي يقوم بهذه المهمة و الذي يجب أن يسبق كل دخول إلى منطقة حرجة ضمن البرنامج يدعى بجزء الدخول (Entry Section).
 - بعد انتهاء التنفيذ في المنطقة الحرجة (Critical Section) قد يكون هناك حاجة لتنفيذ مجموعة من التعليمات تدعى بجزء الخروج (Exit Section).
 - ما تبقى من النص البرمجي للعملية و هو الجزء الذي لا يتم التعامل فيه مع الذاكرة المشتركة يدعى بالجزء المتبقي (Remainder Section).

مسألة المناطق الحرجة

مسألة المناطق الحرجة (Critical Section Problem)

- أي حل لمسألة المنطقة الحرجة يجب أن يحقق مجموعة من المتطلبات:

١- الإقصاء التبادلي (Mutual Exclusion): لا يمكن لأي عمليتين الوجود في نفس الوقت في منطقتهما الحرجة.

٢- التقدم (Progress): لا تستطيع أي عملية تعمل خارج منطقتها الحرجة إيقاف العمليات الأخرى. إذا لم تكن أي عملية في منطقتها الحرجة و هناك عمليات ترغب بالدخول إلى منطقتها الحرجة فعندئذ فقط تلك العمليات التي ليست موجودة في الجزء المتبقي من نصها البرمجي هي التي يجب أخذها بعين الاعتبار لاختيار العملية التي ستدخل إلى منطقتها الحرجة.

٣- الانتظار المحدود (Bounded Waiting): لا تحتاج أي عملية لأن تنتظر للأبد كي تدخل منطقتها الحرجة. أي أنه يجب أن يكون هناك حد لعدد المرات التي تدخل فيها العمليات إلى مناطقها الحرجة بعد أن تقوم عملية ما بطلب الدخول إلى منطقتها الحرجة و قبل تمكنها من الدخول.

بعض الأمثلة التي تحدث فيها مشاكل نتيجة للإيقاف غير المتوقع لبعض العمليات بواسطة المجدول.

1- مشكلة تعديل ملف على مخدم الملفات:

إفترض أن لدينا شبكة حواسيب حيث يتم تخزين كل ملفات المستخدمين في خادم الملفات. إذا كان هنالك مستخدمين يريدان تعديل ملف مشترك بينهما في نفس الوقت، سيقوم كل مستخدم بفتح الملف في جهازه (نسخة من الملف)، ثم يقوم كل منهما بتعديل نسخته، هذا التعديل لن يتم على الملف الأصلي بالخادم وإنما في نسخة كل مستخدم. وعندما يقوم المستخدم بالحفظ سيغير ذلك في الملف الأصلي. المشكلة هي أنه عندما يقوم المستخدم الثاني بحفظ نسخة ملفه في الخادم سيكتب فوق تعديلات المستخدم الذي حفظ ملفه أولاً (overwrite). هذه المشكلة ينتج عنها مخرجات خاطئة.

الصحيح هو أنه عندما تعمل عمليتان في وقت واحد يجب أن تكون النتيجة متشابهة بغض النظر أي عملية أنهت أول. ولكن عندما يؤثر ترتيب تنفيذ العمليات في النتيجة يسمى هذا حالة السباق (race condition). لأن ذلك يمثل سباق بين العمليات لنعرف من ينتهي أولاً.

مشاكل النزاع:

بعض الأمثلة التي تحدث فيها مشاكل نتيجة للإيقاف غير المتوقع لبعض العمليات بواسطة المجدول.

2- مشكلة نظام الحجز الموزع:

إذا كان هنالك نظام حجز على خطوط طيران معينة، يعمل على عدة فروع. وكان هنالك مقعد واحد فارغ في رحلة ما، وجاء عميل لحجز هذا المقعد في الفرع أ، وفي نفس الوقت كان هنالك عميل بالفرع ب، يريد حجز نفس المقعد، وقام الموظفان في الفرعين بعملية طلب حجز للمقعد في نفس الوقت، ماذا يحدث ؟

العملية الأولى في الفرع أ قامت بالتأكد من وجود مقعد فارغ، وهذا ما فعلته أيضا العملية التي نفذت في الفرع ب، فالعمليتان قامتا باختبار وجود مقعد فارغ، ثم أجاب النظام للعمليتين "بنعم يوجد مقعد واحد فارغ"، في هذه الإثناء قام المجدول (بنظام التشغيل) بتوقيف إحدى العمليتين لسبب ما، فقامت العملية الأخرى بحجز المقعد، ثم بعد فك توقيف العملية الأولى ستكمل عملها وتنفذ الأمر الذي يلي اختبار وجود مقعد فارغ (فهي اختبرت المقعد ووجدته فارغا قبل توقيفها). فتقوم العملية بحجز المقعد الذي حجز من قبل، وبهذا يتم حجز المقعد مرتين.

مشاكل النزاع:

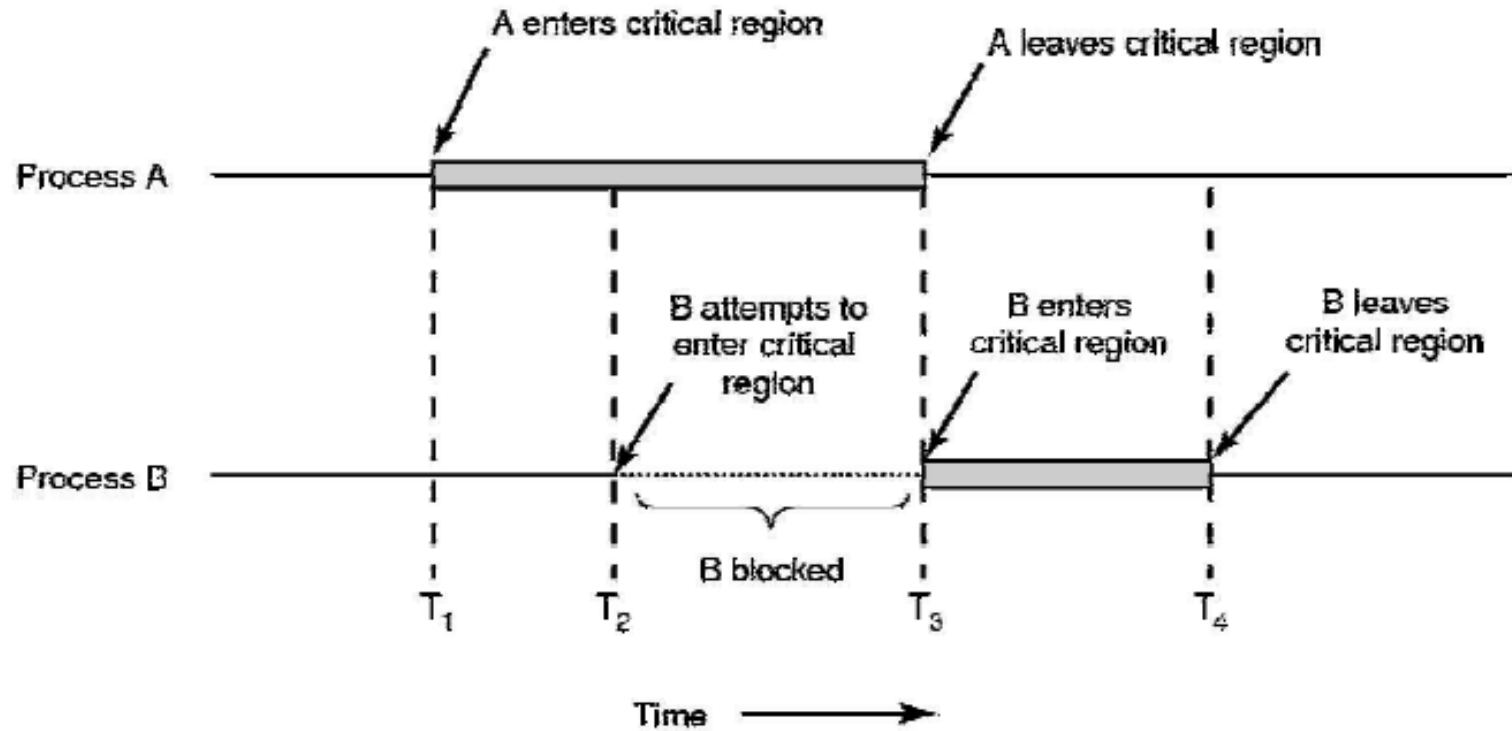
بعض الأمثلة التي تحدث فيها مشاكل نتيجة للإيقاف غير المتوقع لبعض العمليات بواسطة المجدول

3- مشكلة الحساب المشترك:

إذا كان هنالك حساب مفتوح بينك ما، وكان هذا الحساب مشترك بين عميلين، فقد يتفق وأن يطلب العميل سحب مبلغ من المال من الحساب المشترك في وقت واحد من فرعين مختلفين. هنا ستنفذ عمليتين على الحساب المشترك، حيث ستقوم العمليتين باختبار الرصيد (نفترض أنه كان 1500 دولار)، ثم إذا أوقفت إحدى العمليتين بواسطة المجدول لسبب ما، ونفذت العملية الأخرى سحب المبلغ المطلوب (مثلا 100 دولار)، سيكون الرصيد الآن 1400 دولار. بعد أن يتم فك حجز العملية الثانية ستواصل من بعد آخر أمر كانت قد نفذته، وهو اختبار الرصيد (1500 دولار)، وستنفذ السحب ونخصم المبلغ من الرصيد (مثلا إذا كان المبلغ المراد سحبه هو 200 دولار، فسيكون الرصيد المتبقي هو (1300 دولار). وتعتبر هذه العملية خطأ، ولو عمل البنك بهذه الطريقة سيغلق أبوابه قريبا.

النتيجة الخاطئة التي وصلنا إليها كان سببها الوصول غير المنظم لقاعدة البيانات وإجراء تعديل على الحساب المشترك بصورة غير منسقة (غير تزامني) مما نتج عنه خطأ يسمى حالة السباق (race condition). حل مشكلة مثل هذه يتم بالتأكد من أن عملية واحدة فقط في لحظة معينة تقوم بتعديل البيانات المشتركة.

الإقصاء التبادلي باستعمال منطقة حرجية



طرق التعامل مع حالات السباق Managing Race Condition

- تعطيل المقاطعات (Interrupt disabling)
- إقصاء تبادلي مع انتظار شاغل (Mutual Exclusion with Busy Waiting)
- تناوب بين عمليتين (Strict Alternation)
- حل بيترسون (Peterson's Solution)
- تعليمات غير قابلة للتجزئة (Atomic operations)
- إقصاء تبادلي دون انتظار شاغل (Mutual Exclusion without Busy Waiting)
- تنويم و إيقاظ (Sleep and wakeup)
- مسيرات (Semaphore)
- مراقبات (Monitor)

الإننتظار الشاغل يعني إضاعة وقت المعالج

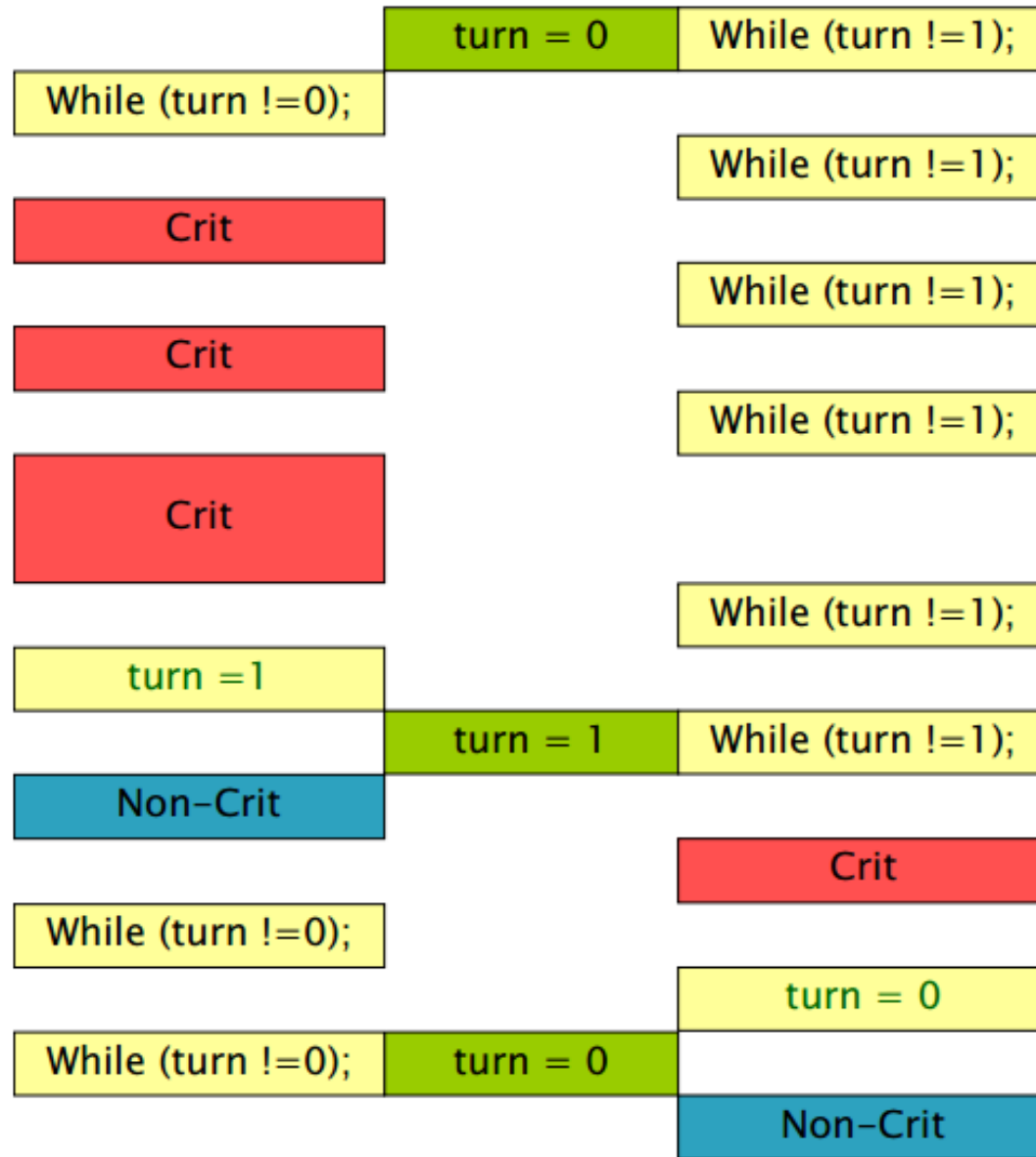
التناوب (Strict Alternation) بين عمليتين (فقط)

```
while (TRUE) {
    while (turn != 0)      /* loop */ ;
    critical_region();
    turn = 1;
    noncritical_region();
}
```

```
while (TRUE) {
    while (turn != 1)      /* loop */ ;
    critical_region();
    turn = 0;
    noncritical_region();
}
```

0	turn = 0	1
While (turn !=0);		While (turn !=1);
Crit		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
turn = 1		While (turn !=1);
Non-Crit	turn = 1	While (turn !=1);
While (turn !=0);		Crit
While (turn !=0);		turn = 0
While (turn !=0);	turn = 0	Non-Crit
Crit		While (turn !=1);
		While (turn !=1);

- تتناوب العمليتان على الدخول إلى مناطقيهما الحرجة
- التناوب في الحصول على المورد المشترك (Shard resource)



التناوب بين عمليتين Strict Alternation

- يقوم معالج واحد بتنفيذ المسارين (Execution path) سواء كانا عمليتين (Processes) أم مسلكين (Threads)

التناوب (Strict Alternation) بين عمليتين (فقط)

0

1

	turn = 0	
While (turn !=0);		While (turn !=1);
Crit		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
turn =1		While (turn !=1);
Non-Crit	turn = 1	While (turn !=1);
While (turn !=0);		Crit
While (turn !=0);		turn = 0
While (turn !=0);	turn = 0	Non-Crit
Crit		While (turn !=1);

- إنتظار شاغل فيه
هدر لوقت المعالج
- يشغل المعالج
بالتحقق من الشرط
مرات عديدة حتى
يسمح للعملية 1
بالدخول إلى منطقتها
الحرّة

التناوب (Strict Alternation) بين عمليتين (فقط)

0

1

While (turn !=0);	turn = 0	While (turn !=1);
Crit		While (turn !=1);
turn = 1		While (turn !=1);
Non-Crit	turn = 1	While (turn !=1);
		Crit
		turn = 0
	turn = 0	Non-Crit
		While (turn !=1);
While (turn !=0);		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
		While (turn !=1);
Crit		While (turn !=1);
turn = 1		While (turn !=1);
Non-Crit	turn = 1	While (turn !=1);

• حل سيء عندما تكون

إحدى العمليتين أبطأ

كثيراً من الأخرى

• لا يحقق شرط التقدم،

لأن العملية 0 مثلاً

يمكن أن تعرقل عمل

العملية 1 من خارج

منطقتها الحرجة في

حال كون زمن تنفيذ

هذه المنطقة غير

الحرجة طويل نسبياً

حل بيترسون (Peterson's Solution)

do {

```
flag[ i ] = TRUE;
turn = j;
while (flag[ j ] && turn == j);
```

critical section

```
flag[ i ] = FALSE;
```

remainder section

} while (TRUE);

- يمكن تطبيق حل بيترسون فقط في حالة عمليتين لديهما ذاكرة مشتركة.
- لتحقيق هذا الحل نقوم بتعريف متحولين مشتركين بين العمليتين:
- يحدد المتحول **int turn** العملية التي لها الدور للدخول إلى منطقها الحرجة.
- المصفوفة **boolean flag[2]** تستخدم لتحديد فيما إذا كانت العملية جاهزة لدخول إلى منطقها الحرجة.
- بمعنى **flag[i] = true** تعني أن العملية **i** جاهزة للدخول إلى منطقها الحرجة.

حل بيترسون (Peterson's Solution)

do {
 flag[1]=true;
 turn=2;
 while (flag[2] && turn=2) ;
 critical_region();
 flag[1]=false;
 noncritical_region();
 } while (true);

1

do {
 flag[2]=true;
 turn=1;
 while (flag[1] && turn=1) ;
 critical_region();
 flag[2]=false;
 noncritical_region();
 } while (true);

2

- يحقق حل بيترسون الإقصاء التبادلي.
- يحقق حل بيترسون شرط التقدم (Progress).
- يحقق حل بيترسون شرط الانتظار المحدود.

do {
 flag[1]=true; turn=2;
 while (flag[2] && turn=2) ;
 critical_region();
 flag[1]=false;
 noncritical_region();
} while (true);

1

do {
 flag[2]=true; turn=1;
 while (flag[1] && turn=1) ;
 critical_region();
 flag[2]=false;
 noncritical_region();
} while (true);

2

	flag[1]	flag[2]	turn	
		true		flag[2]=true
flag[1]=true	true			
			1	turn=1;
turn=2			2	
				while(flag[1] && turn=1);
while(flag[2] && turn=2);				
				critical_region()
while(flag[2] && turn=2);				
				critical_region()
while(flag[2] && turn=2);				
				critical_region()
while(flag[2] && turn=2);	true	true	2	

```
do {
    flag[1]=true;   turn=2;
    while ( flag[2] && turn=2) ;
    critical_region();
    flag[1]=false;
    noncritical_region();
} while (true);
```

1

```
do {
    flag[2]=true;   turn=1;
    while ( flag[1] && turn=1) ;
    critical_region();
    flag[2]=false;
    noncritical_region();
} while (true);
```

2

	flag[1]	flag[2]	turn	
	true	true	2	critical_region()
while(flag[2] && turn=2);				
		false		flag[2]=false
while(flag[2] && turn=2);				
				noncritical_region()
critical_region()				
				noncritical_region()
flag[1]=false	false			
				noncritical_region()
noncritical_region()				
				noncritical_region()
flag[1]=true; turn =2	true	false	2	


```
do {
    flag[1]=true;   turn=2;
    while ( flag[2] && turn=2) ;
    critical_region0;
    flag[1]=false;
    noncritical_region0;
} while (true);
```

1

```
do {
    flag[2]=true;   turn=1;
    while ( flag[1] && turn=1) ;
    critical_region0;
    flag[2]=false;
    noncritical_region0;
} while (true);
```

2

	flag[1]	flag[2]	turn	
	true	false	2	noncritical_region()
while(flag[2] && turn=2);				
		false		noncritical_region()
critical_region()				
				noncritical_region()
flag[1]=false	false			
				noncritical_region()
noncritical_region()				
				noncritical_region()
flag[1]=true; turn =2				
				noncritical_region()
while(flag[2] && turn=2);				

حل بيترسون (Peterson's Solution)

- انتقل السباق من الحصول على المورد في المنطقة الحرجة

إلى سباق الحصول على الدور `turn`

- القيمة المحفوظة في المتحول `turn` تتبع للتسلسل الفعلي لتنفيذ التعليمات في المسارات المختلفة

- لن يحدث خلل في البيانات بهذه الطريقة لأن `turn` يمكن أن يأخذ إحدى قيمتين، و قيمته ستكون

هي القيمة التي اختارتها العملية الأخيرة التي قامت بتعديل قيمة الـ `turn`

- أي أن السباق على `turn` لن يسبب خلل في البيانات الفعلية

- لكن الحل بهذه الطريقة لا يضمن أن تكون العملية التي حصلت على المورد المشترك أولاً هي

بالفعل العملية التي طلبت الحصول على المورد المشترك أولاً

- لتحقيق ذلك نلجأ لاستخدام تعليمات غير القابلة للتجزئة للحصول على الدور

حلول المزامنة المدعومة من قبل العتاد

- يتوفر في العتاد الحاسوبي حاليا تعليمات غير قابلة للتجزئة تدعى (Atomic Instructions) أي أنها تعليمات لا يمكن مقاطعة تنفيذها قبل انتهائها
 - التعليمة (TestAndSet) و التي تقوم بتفحص محتوى كلمة في الذاكرة و تعديله
 - التعليمة (Swap) التي تقوم بالمبادلة بين قيمتي كلمتين في الذاكرة

```
boolean TestAndSet (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

```
void Swap (boolean *a, boolean *b)
{
    boolean temp = *a;
    *a = *b;
    *b = temp;
}
```

حلول المزامنة المدعومة من قبل العتاد

- الحلول باستخدام التعليمات غير قابلة للتجزئة تأخذ الشكل العام التالي

do {

acquire lock

critical section

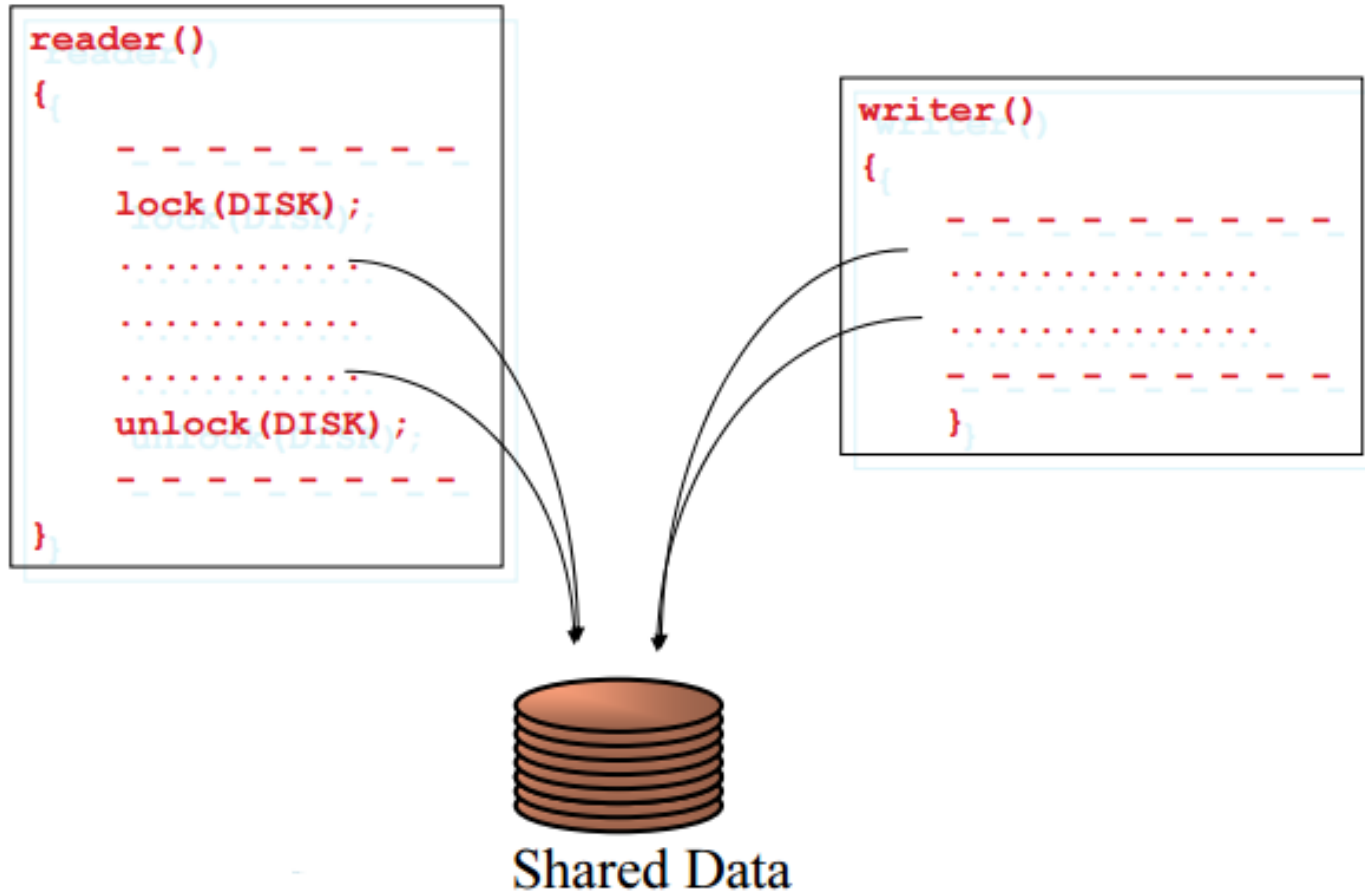
release lock

remainder section

} while (TRUE);

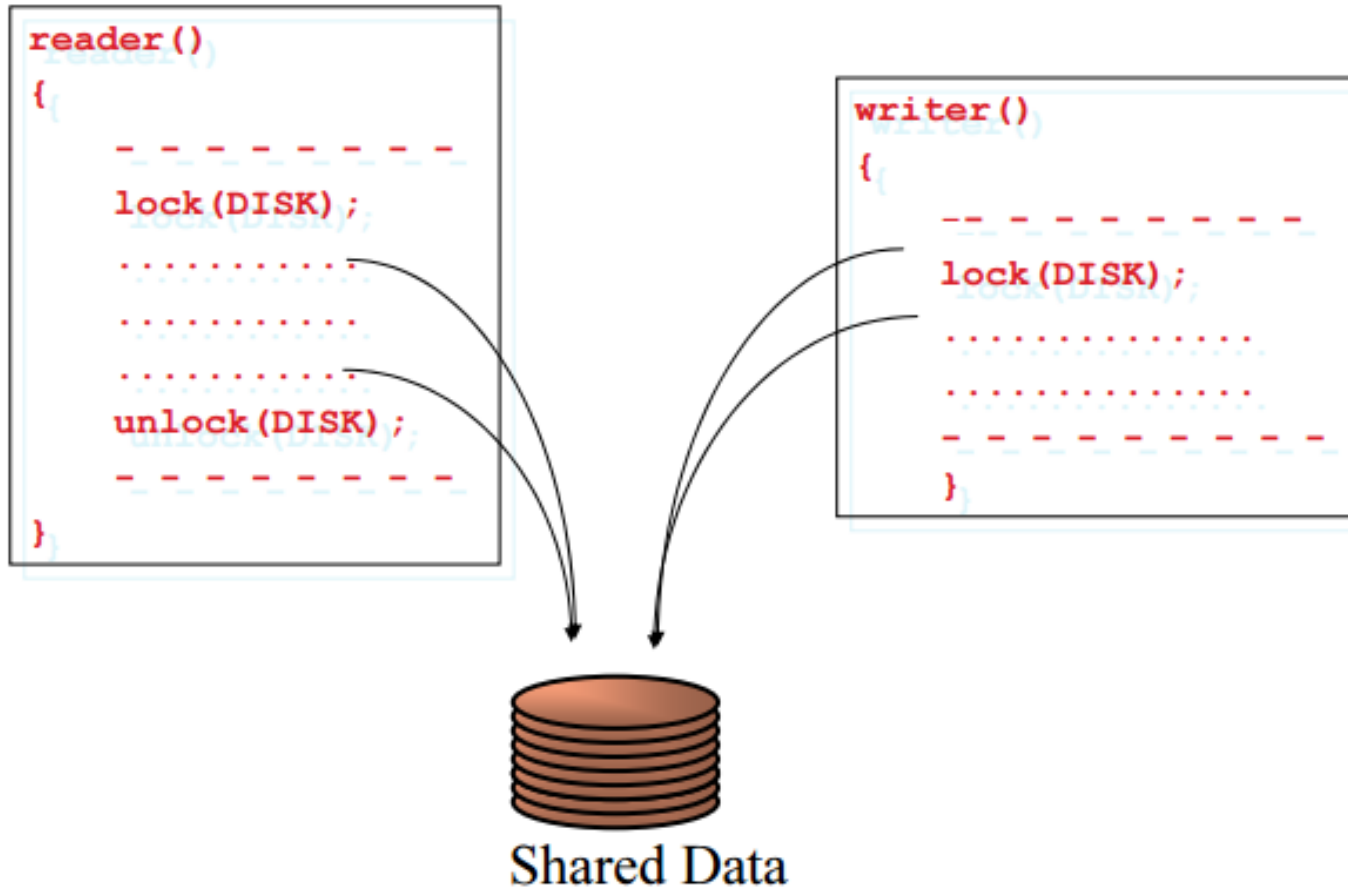
حلول المزامنة المدعومة من قبل النظام

- المبرمج السيئ يكتب برامج لها الشكل



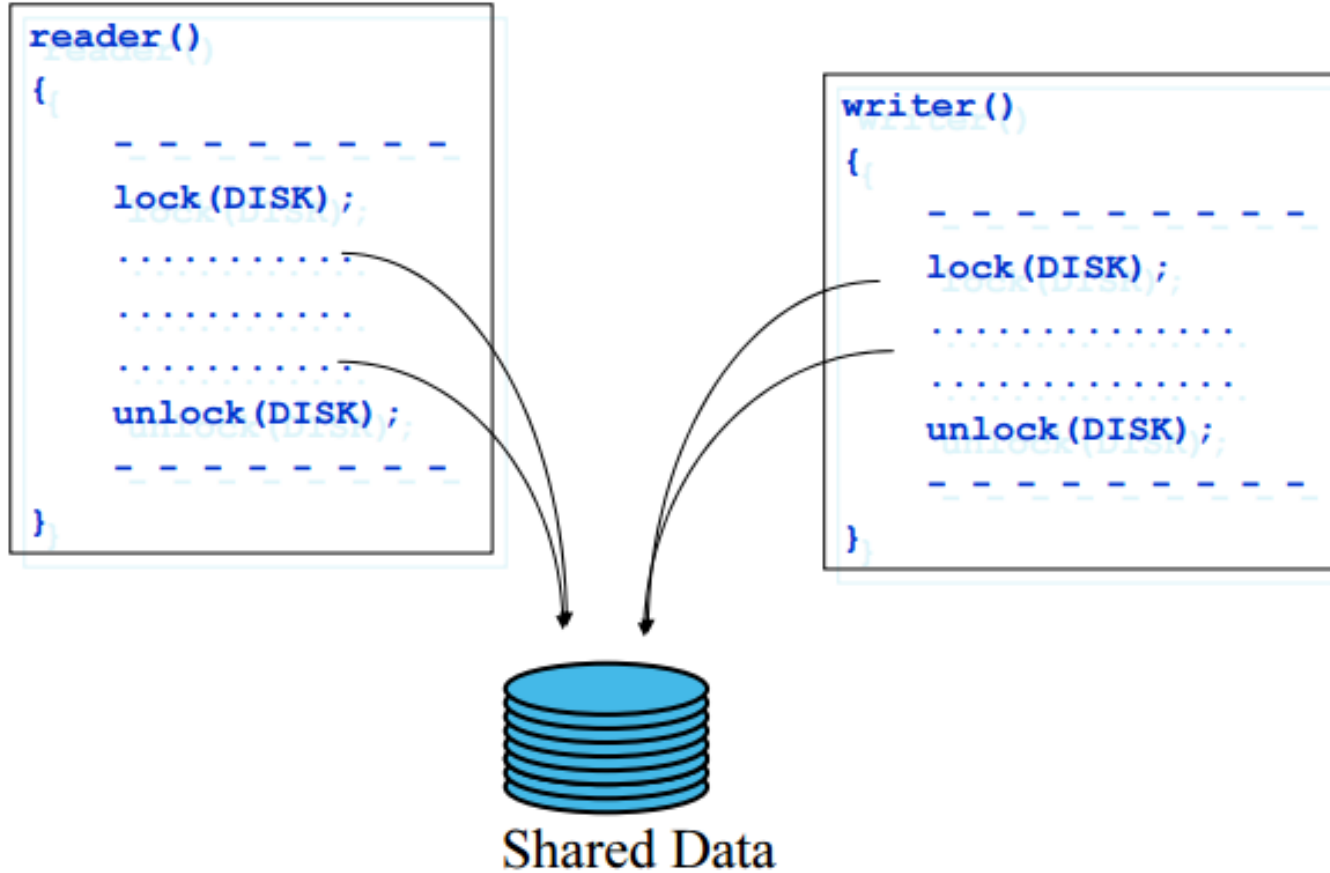
حلول المزامنة المدعومة من قبل النظام

- المبرمج النعسان يكتب برامجها لها الشكل



حلول المزامنة المدعومة من قبل العتاد

• المبرمج الجيد يكتب برامجها لها الشكل



حلول المزامنة المدعومة من قبل العتاد

حل مسألة المنطقة الحرجة باستخدام التعليمة (TestAndSet)

- نقوم بتعريف المتحول lock من النوع boolean و الذي يأخذ القيمة الابتدائية false

do {

while (TestAndSet (&lock))
; // do nothing

// critical section

lock = FALSE;

// remainder section

} while (TRUE);

- إن استعمال التعليمات غير القابلة للتجزئة يمكن من تحقيق شرط الإقصاء التبادلي ببساطة بين N عملية

إلا أنها لا تضمن تحقيق شرط الانتظار المحدود

لذلك نستفيد من الخبرة المكتسبة من حل

بيترسون في تعديل هذا الحل بحيث يتحقق

شرط الانتظار المحدود

```

do {
    waiting[ i ] = TRUE;
    key = TRUE;
    while (waiting[ i ] && key)
        key = TestAndSet(&lock);
    waiting[ i ] = FALSE;
        // critical section
    j = (i + 1) % n;
    while ((j != i) && !waiting[ j ])
        j = (j + 1) % n;
    if (j == i)
        lock = FALSE;
    else
        waiting[ j ] = FALSE;
        // remainder section
} while (TRUE);

```

- حل مسألة المنطقة الحرجة باستخدام التعليمة TestAndSet مع تحقيق الانتظار المحدود لـ N عملية
- Waiting[i]=true : العملية i تنتظر دورها في استعمال المورد المشترك
- Waiting[i]=false : العملية i لم تعد تنتظر، أي حان دورها في استعمال المورد المشترك
- واحدة فقط من العمليات تستعمل المورد المشترك في لحظة ما
- واحدة فقط لها Waiting[i]=false أما البقية فلهم Waiting[i]=true

do {

waiting[i] = TRUE;

key = TRUE;

while (waiting[i] && key)

key = TestAndSet(&lock);

waiting[i] = FALSE;

// critical section

j = (i + 1) % n;

while ((j != i) && !waiting[j])

j = (j + 1) % n;

if (j == i)

lock = FALSE;

else

waiting[j] = FALSE;

// remainder section

} while (TRUE);

• حل مسألة المنطقة الحرجة

باستخدام التعليمة TestAndSet

مع تحقيق الانتظار المحدود

لـ N عملية

• key=true : العملية تنتظر السماح لها

باستعمال مفتاحها لفتح قفل (Lock)

المنطقة الحرجة

• key=false : العملية لم تعد تنتظر،

و يمكنها استعمال مفتاحها على

قفل المنطقة الحرجة (Lock)

• واحدة فقط من العمليات يمكنها استعمال

مفتاحها على القفل (Lock) في لحظة ما

do {

waiting[i] = TRUE;

key = TRUE;

while (waiting[i] && key)

key = TestAndSet(&lock);

waiting[i] = FALSE;

// critical section

j = (i + 1) % n;

while ((j != i) && !waiting[j])

j = (j + 1) % n;

if (j == i)

lock = FALSE;

else

waiting[j] = FALSE;

// remainder section

} while (TRUE);

• حل مسألة المنطقة الحرجة

باستخدام التعليمة TestAndSet

مع تحقيق الانتظار المحدود

لـ N عملية

• القفل (Lock) يبدأ بحالة لا أحد يستعمله

Lock=false

• بمجرد أن تقوم إحدى العمليات باستعمال

مفتاحها على القفل فإنه ينتقل لحالة

Lock=true أي أنه قيد الاستعمال

بواسطة atomic instruction

• عندما تنتهي عملية من استعمال المورد

المشترك فإنها تزيل مفتاحها من القفل معيدة

إياه إلى حالة لا أحد يستعمله Lock=false

do {

waiting[i] = TRUE;

key = TRUE;

while (waiting[i] && key)

key = TestAndSet(&lock);

waiting[i] = FALSE;

// critical section

j = (i + 1) % n;

while ((j != i) && !waiting[j])

j = (j + 1) % n;

if (j == i)

lock = FALSE;

else

waiting[j] = FALSE;

// remainder section

} while (TRUE);

• حل مسألة المنطقة الحرجة

باستخدام التعليمة TestAndSet

مع تحقيق الانتظار المحدود

لـ N عملية

• يحقق هذا الحل شرط الإقصاء التبادلي

• نلاحظ أنه يمكن لعملية P_i الدخول إلى منطقتها

الحرجة فقط في حال كان

• إما key=false

• فقط عند تنفيذ Key = TestAndSet

للمرة الأولى من قبل إحدى العمليات

• بقية العمليات ستنتظر حتى تتاح لهما

امكانية استعمال القفل لاحقا

• أو Waiting[i]=false

do {

waiting[i] = TRUE;

key = TRUE;

while (waiting[i] && key)

key = TestAndSet(&lock);

waiting[i] = FALSE;

// critical section

j = (i + 1) % n;

while ((j != i) && !waiting[j])

j = (j + 1) % n;

if (j == i)

lock = FALSE;

else

waiting[j] = FALSE;

// remainder section

} while (TRUE);

• حل مسألة المنطقة الحرجة

باستخدام التعليمة TestAndSet

مع تحقيق الانتظار المحدود

لـ N عملية

• يحقق هذا الحل شرط الإقصاء التبادلي

• نلاحظ أنه يمكن لعملية P_i الدخول إلى منطقتها

الحرجة فقط في حال كان

• إما key=false

• أو Waiting[i]=false

• فقط عندما تترك إحدى العمليات

منطقتها الحرجة

do {

waiting[i] = TRUE;

key = TRUE;

while (waiting[i] && key)

key = TestAndSet(&lock);

waiting[i] = FALSE;

// critical section

j = (i + 1) % n;

while ((j != i) && !waiting[j])

j = (j + 1) % n;

if (j == i)

lock = FALSE;

else

waiting[j] = FALSE;

// remainder section

} while (TRUE);

• حل مسألة المنطقة الحرجة

باستخدام التعليمة TestAndSet

مع تحقيق الانتظار المحدود

لـ N عملية

• يحقق هذا الحل شرط التقدم

• لأن أي عملية تنتهي من تنفيذ منطقها الحرجة فإنها

• إما أن تجعل lock=false

• أو Waiting[i]=false

سامحة بذلك لعملية أخرى أن تدخل إلى

منطقها الحرجة و تتقدم لاحقا

do {

waiting[i] = TRUE;

key = TRUE;

while (waiting[i] && key)

key = TestAndSet(&lock);

waiting[i] = FALSE;

// critical section

j = (i + 1) % n;

while ((j != i) && !waiting[j])

j = (j + 1) % n;

if (j == i)

lock = FALSE;

else

waiting[j] = FALSE;

// remainder section

} while (TRUE);

• حل مسألة المنطقة الحرجة

باستخدام التعليمة TestAndSet

مع تحقيق الانتظار المحدود

لـ N عملية

• يحقق هذا الحل شرط الانتظار المحدود

• لأن أي عملية i عندما تنتهي من تنفيذ منطقتها

الحرجة فإنها ستعطي الدور لإحدى العمليات

التي تنتظر السماح بالدخول إلى منطقتها الحرجة

(Waiting[i]=true)

• يتم اختيار العملية بشكل دائري

(i+1,..., n , 0 , 1 , ..., i-1)

أي أن أي عملية ستنتظر n-1 مرة على الأكثر

حتى يحين دورها


```

do {
    waiting[ i ] = TRUE;
    key = TRUE;
    while (waiting[ i ] && key)
        key = TestAndSet(&lock);
    waiting[ i ] = FALSE;
        // critical section
    j = (i + 1) % n;
    while ((j != i) && !waiting[ j ])
        j = (j + 1) % n;
    if (j == i)
        lock = FALSE;
    else
        waiting[ j ] = FALSE;
        // remainder section
} while (TRUE);

```

• حل مسألة المنطقة الحرجة
 باستخدام التعليمة TestAndSet
 مع تحقيق الانتظار المحدود
 لـ N عملية

• يستخدم هذا الحل الانتظار الشاغل

حلول المزامنة المدعومة من قبل العتاد

حل مسألة المنطقة الحرجة باستخدام التعليمة (Swap)

- نقوم بتعريف المتحول lock من النوع boolean مشترك بين العمليات و الذي يأخذ القيمة الابتدائية false، كما نعرف متحول محلي لكل عملية نسميه key من النوع boolean

```
do {  
    key = TRUE;  
    while ( key == TRUE)  
        Swap (&lock, &key );  
  
    // critical section  
  
    lock = FALSE;  
  
    // remainder section  
  
} while (TRUE);
```

- key = true تعني أن العملية ترغب بالانتظار حتى

يحين دورها في الحصول على القفل الذي يسمح بالدخول إلى منطقتها الحرجة و الحصول على المورد المشترك

- لا يضمن هذا الحل تحقيق شرط الانتظار المحدود !

- على غرار الحل باستعمال TestAndSet يمكن تعديل

هذا الحل بحيث يحقق شروط مسألة المنطقة الحرجة الثلاثة

حلول المزامنة المدعومة من قبل العتاد

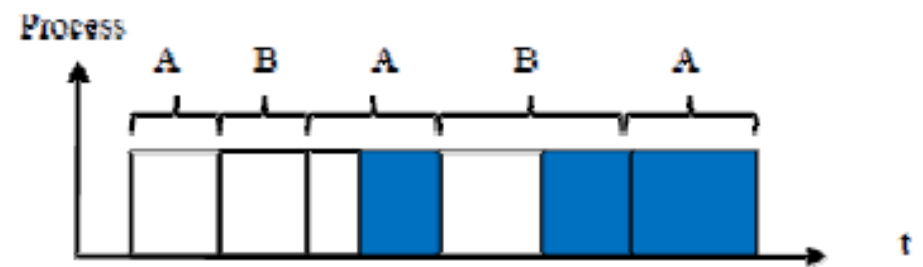
تعطيل المقاطعات (Interrupt disabling)

- يمكن إلغاء المقاطعات عندما تدخل عملية ما إلى منطقتها الحرجة و من ثم إعادة تفعيل المقاطعات عند خروجها من المنطقة الحرجة
- عند تعطيل المقاطعات فإن المعالج لن يتمكن من مقاطعة العملية قبل انتهاءها من تنفيذ منطقتها الحرجة و خروجها منها لأنه لا يمكن له القيام ب switch context في حال تعطيل المقاطعات
- تعطيل المقاطعات في حالات الأنظمة ذات المعالج الوحيد يضمن أن العملية التي تستخدم موردا مشتركا ستقوم باستخدامه وحدها دون أن تتمكن أي عملية أخرى من مشاركتها بهذا المورد
- يؤدي تعطيل المقاطعات إلى إضعاف أداء خوارزميات الجدولة بشكل ملحوظ
- يمكن للعملية أن تبقى في منطقها الحرجة لفترة طويلة من الزمن معطلة بذلك جميع المقاطعات مما قد يتسبب في فشل النظام ككل (Crash)
- لا يمكن تطبيق هذا الأسلوب في حالة الأنظمة ذات المعالجات المتعددة

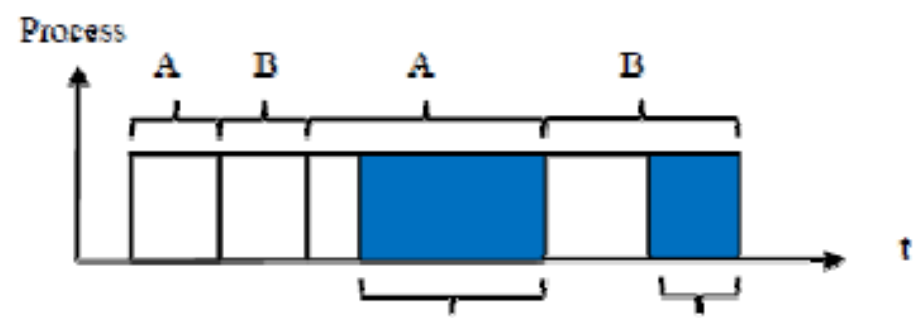
حلول المزامنة المدعومة من قبل العتاد

تعطيل المقاطعات (Interrupt disabling)

Scheduling of two processes A, B accessing a critical section without interrupt disabling



Scheduling of two processes A, B accessing a critical section with interrupt disabling



Access a critical section
disable interrupt

Release a critical section
enable interrupt

disable interrupts "can't be B"
disable interrupts "can't be A"

 Correspond to the areas of critical sections

طرق التعامل مع حالات السباق Managing Race Condition

. تعطيل المقاطعات (Interrupt disabling)

. إقصاء تبادلي مع انتظار شاغل (Mutual Exclusion with Busy Waiting)

. تناوب بين عمليتين (Strict Alternation)

. حل بيترسون (Peterson's Solution)

. تعليمات غير قابلة للتجزئة (Atomic operations)

الطريقة	المنهجية	النمط
Interrupt disabling	Interrupt disabling	Hardware
Atomic instructions	Busy waiting	
Peterson Algorithm		Software

طرق التعامل مع حالات السباق Managing Race Condition

• تعطيل المقاطعات (Interrupt disabling)

• إقصاء تبادلي مع انتظار شاغل (Mutual Exclusion with Busy Waiting)

• تناوب بين عمليتين (Strict Alternation)

• حل بيترسون (Peterson's Solution)

• تعليمات غير قابلة للتجزئة (Atomic operations)

• إقصاء تبادلي دون انتظار شاغل (Mutual Exclusion without Busy Waiting)

• تنويم و إيقاظ (Sleep and wakeup)

• مسيرات (Semaphore)

• مراقبات (Monitor)

الإنتظار الشاغل يعني إضاعة وقت المعالج

تنويم و إيقاظ (Sleep and wakeup)

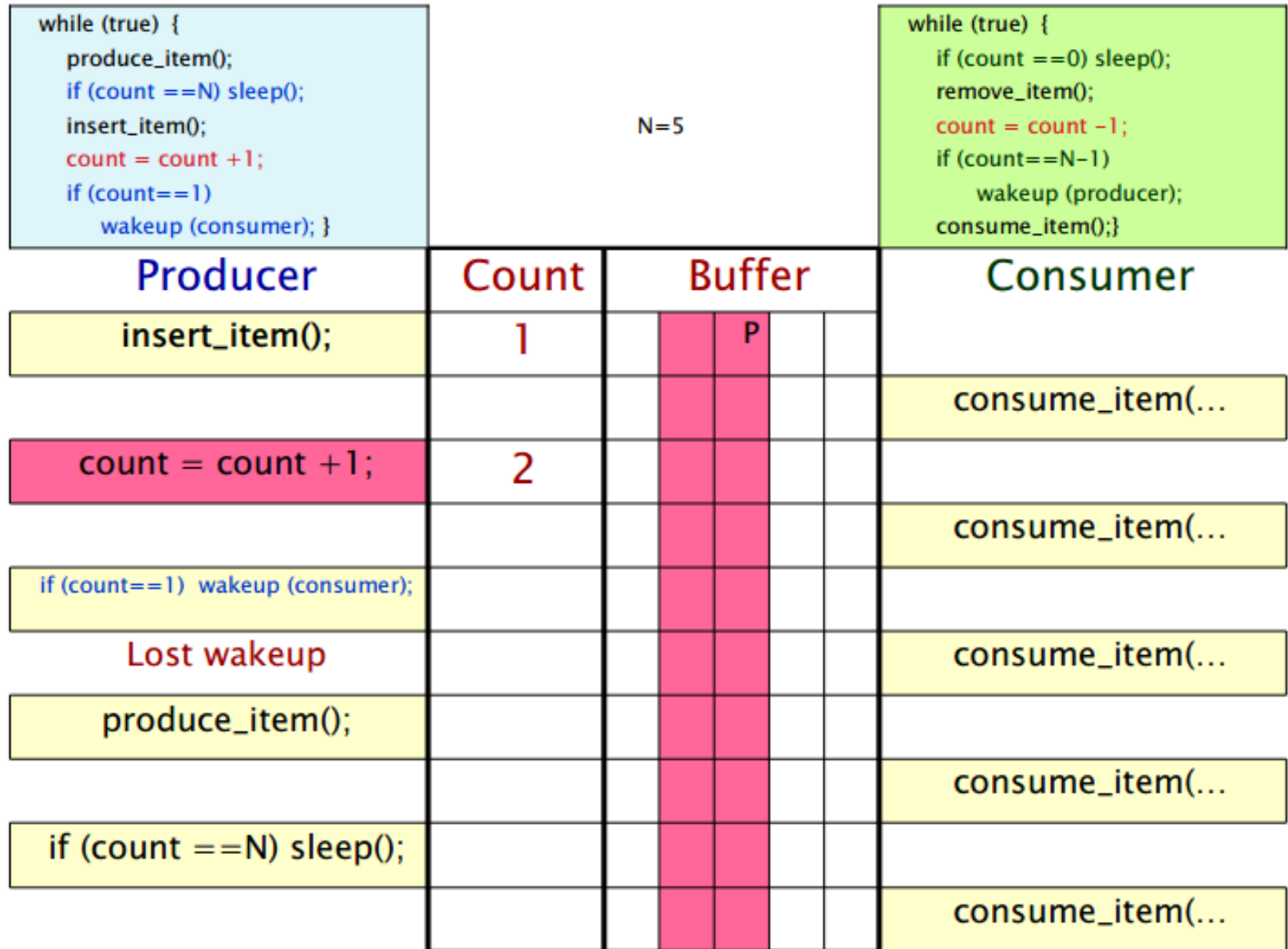
<pre>#define N 100 // Number of slots in buffer int count = 0 ; // Number of items in buffer</pre>	
Producer	
<pre>while (true) { produce_item(); if (count == N) sleep(); insert_item(); count = count + 1; if (count == 1) wakeup (consumer); }</pre>	<pre>repeat forver generate next item if full buffer, go to sleep put item in buffer increment count of items in buffer Was buffer empty ?</pre>

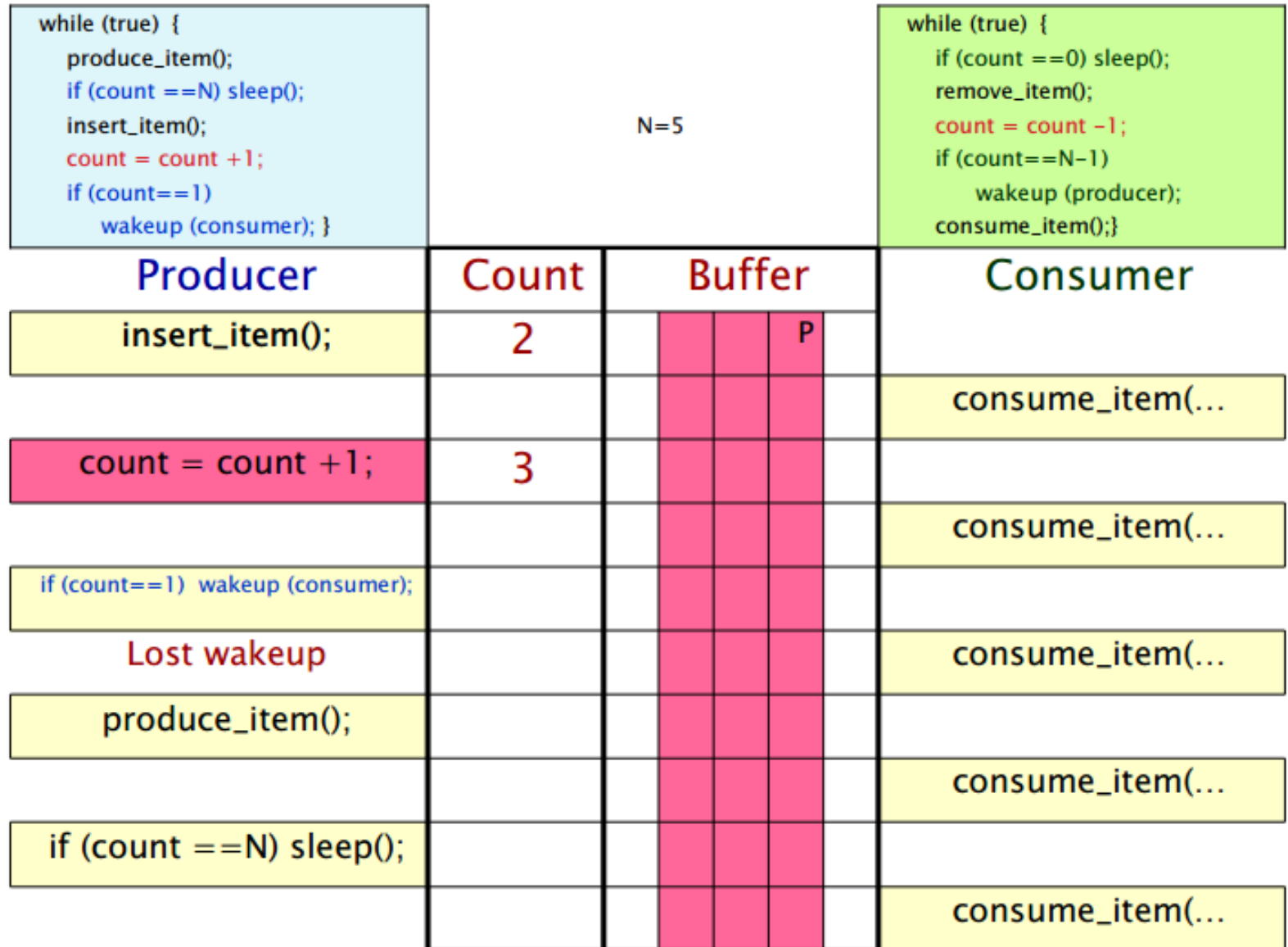
تنويم و إيقاظ (Sleep and wakeup)

<pre>#define N 100 // Number of slots in buffer int count = 0 ; // Number of items in buffer</pre>	
Consumer	
<pre>while (true) { if (count ==0) sleep(); item = remove_item(); count = count -1; if (count==N-1) wakeup (producer); consume_item(item); }</pre>	<pre>repeat forever if empty buffer, go to sleep take item out of buffer decrement count of items in buffer was buffer full ? print item</pre>

<pre>while (true) { produce_item(); if (count ==N) sleep(); insert_item(); count = count +1; if (count==1) wakeup (consumer); }</pre>	N=5					<pre>while (true) { if (count ==0) sleep(); remove_item(); count = count -1; if (count==N-1) wakeup (producer); consume_item(item);}</pre>	
Producer	Count	Buffer					Consumer
	0						if (count ==0) sleep();
insert_item();		P					انتظار غير شاغل
count = count +1;	1						
if (count==1) wakeup (consumer);							
		C					remove_item();
produce_item();							
	0						count = count -1;
if (count ==N) sleep();							
							if (count==N-1) wakeup (producer);
insert_item();	0		P				Lost wakeup

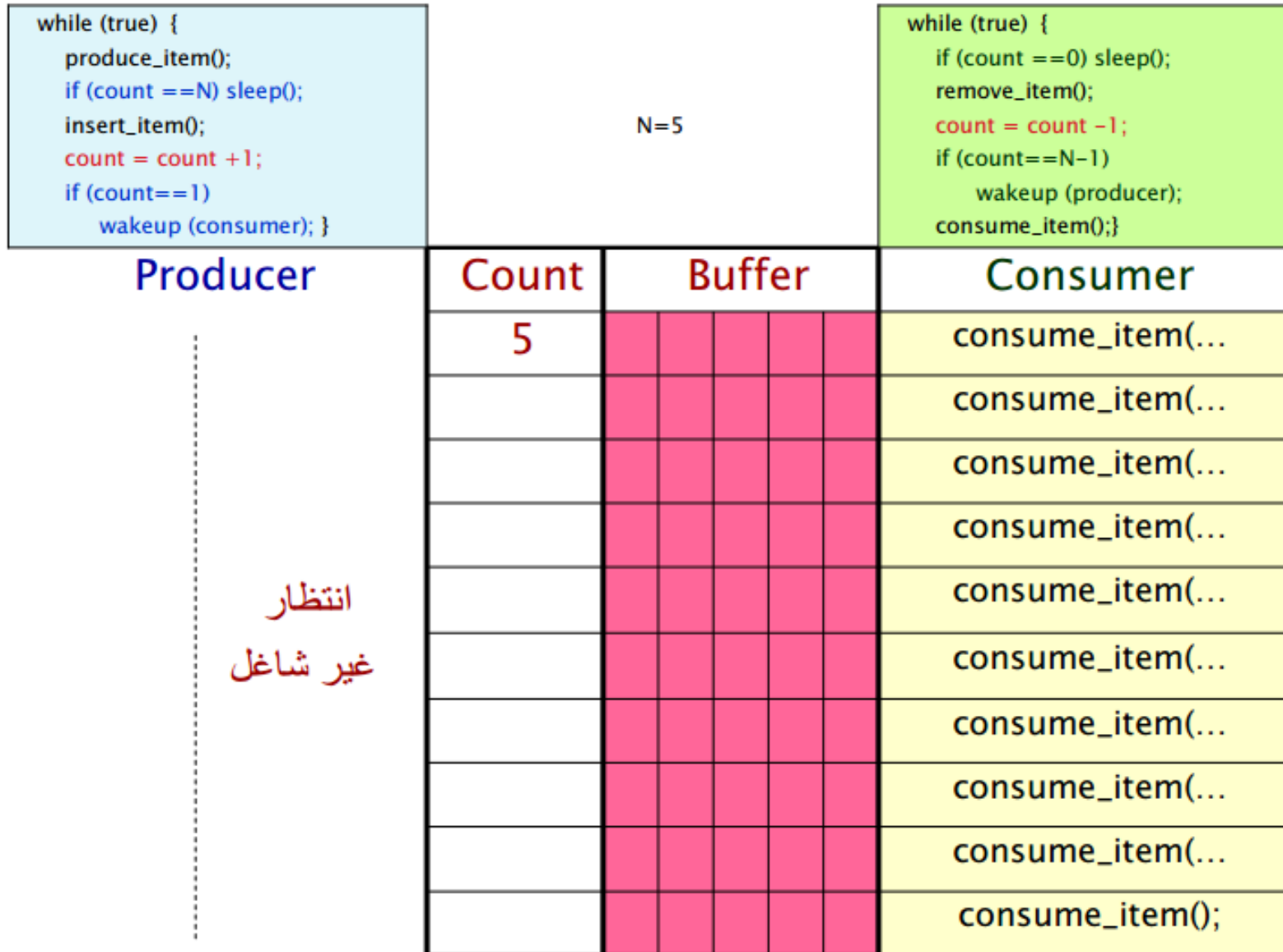
<pre>while (true) { produce_item(); if (count ==N) sleep(); insert_item(); count = count +1; if (count==1) wakeup (consumer); }</pre>	N=5				<pre>while (true) { if (count ==0) sleep(); remove_item(); count = count -1; if (count==N-1) wakeup (producer); consume_item();}</pre>	
Producer	Count	Buffer				Consumer
insert_item();	0	P				
						consume_item(...
count = count +1;	1					
						consume_item(...
if (count==1) wakeup (consumer);						
Lost wakeup						consume_item(...
produce_item();						
						consume_item(...
if (count ==N) sleep();						
						consume_item(...

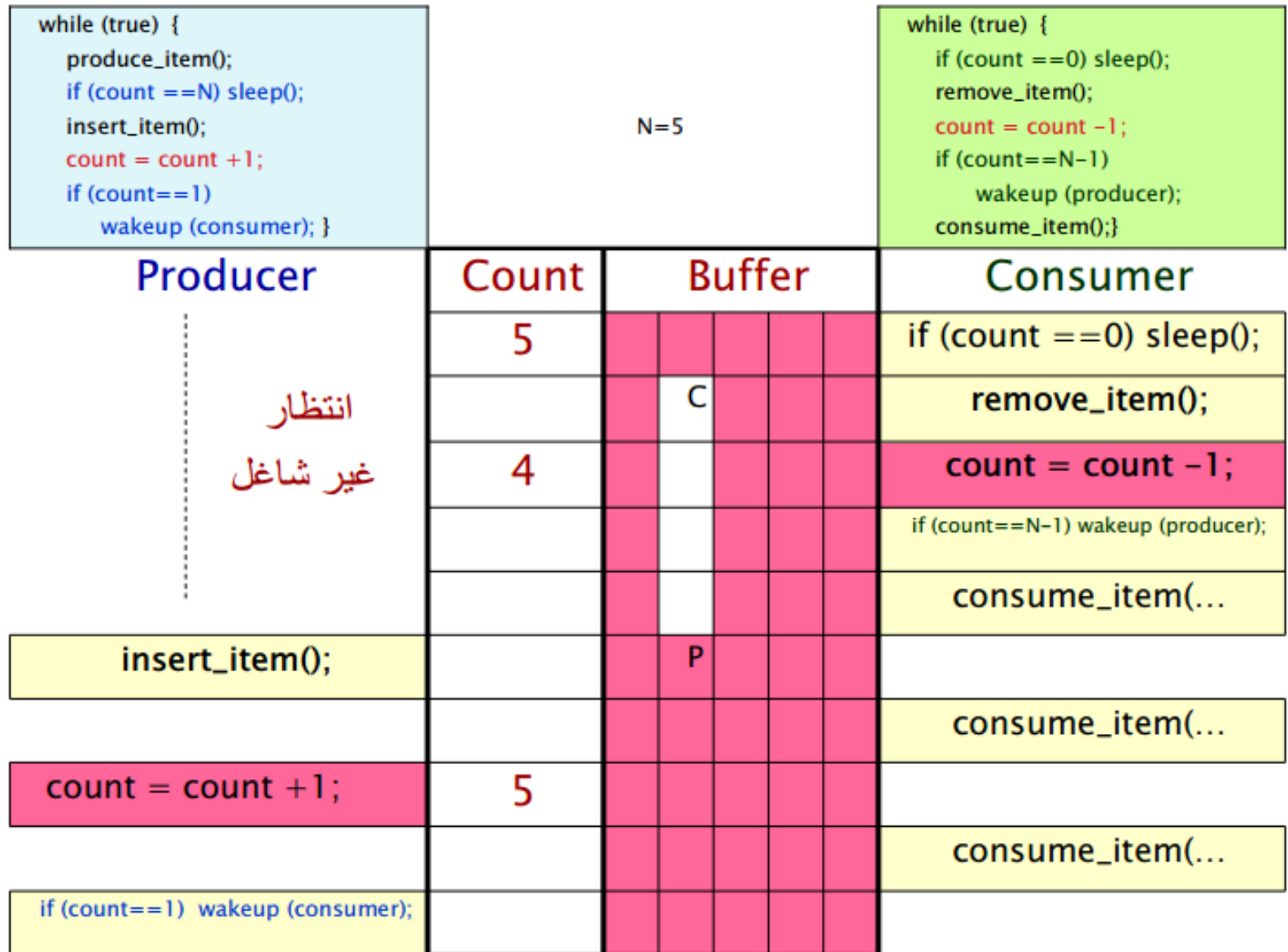


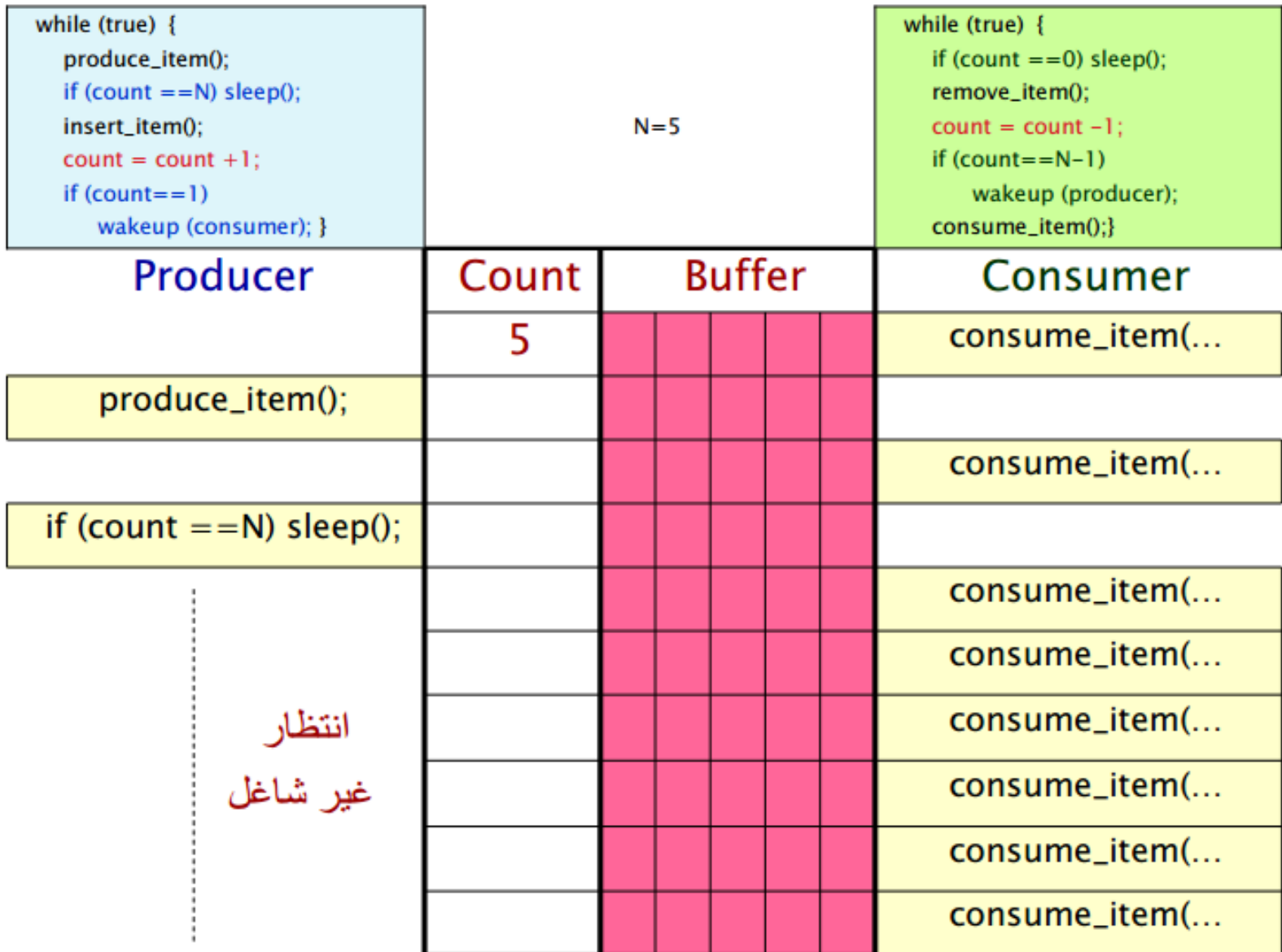


<pre>while (true) { produce_item(); if (count ==N) sleep(); insert_item(); count = count +1; if (count==1) wakeup (consumer); }</pre>	N=5					<pre>while (true) { if (count ==0) sleep(); remove_item(); count = count -1; if (count==N-1) wakeup (producer); consume_item();}</pre>	
Producer	Count	Buffer					Consumer
insert_item();	3					P	
							consume_item(...
count = count +1;	4						
							consume_item(...
if (count==1) wakeup (consumer);							
Lost wakeup							consume_item(...
produce_item();							
							consume_item(...
if (count ==N) sleep();							
							consume_item(...

<pre>while (true) { produce_item(); if (count ==N) sleep(); insert_item(); count = count +1; if (count==1) wakeup (consumer); }</pre>	N=5					<pre>while (true) { if (count ==0) sleep(); remove_item(); count = count -1; if (count==N-1) wakeup (producer); consume_item();}</pre>	
Producer	Count	Buffer					Consumer
insert_item();	4	P					
							consume_item(...
count = count +1;	5						
							consume_item(...
if (count==1) wakeup (consumer);							
Lost wakeup							consume_item(...
produce_item();							
							consume_item(...
if (count ==N) sleep();							
⋮							consume_item(...







المسيرات - Semaphores

- توفر العديد من لغات البرمجة أداة بديلة لتحقيق المزامنة بين العمليات بطريقة بسيطة و فعالة، تدعى هذه الأداة بالمسير (Semaphore).
- المسير (S) هو عبارة عن Flag للإشارة إلى عملية ما أنه لا يسمح لها بالمتابعة و استعمال مورد مشترك (shared resource) لأن عملية أخرى تقوم باستعماله
- يمكن التعامل مع المسير من خلال عمليتين فقط، كلاهما غير قابلة للتجزئة و هما:

wait(S) -

signal(S) -

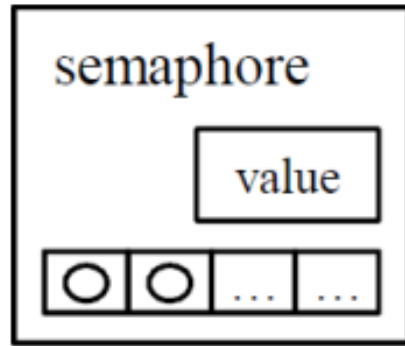
- هناك نوعين من المسيرات:
- المسيرات الثنائية (Binary Semaphore): يأخذ المسير في هذه الحالة إما القيمة 0 أو 1، و تعرف هذه المسيرات عادة باسم (mutex locks).
- العدادات (Counting Semaphore): يكون فيها المسير عبارة عن عدد صحيح يمكن أن يأخذ أي قيمة عددية صحيحة.

المسيرات - Semaphores

بنية المسيرات الداخلية

يمكن بناء المسيرات بطريقة الانتظار الشاغل كمايلي :

```
signal (S) {  
    S++;  
}  
  
wait (S) {  
    while S <= 0  
        ; // no-op  
    S--;  
}
```



المسيرات - Semaphores

بنية المسيرات الداخلية

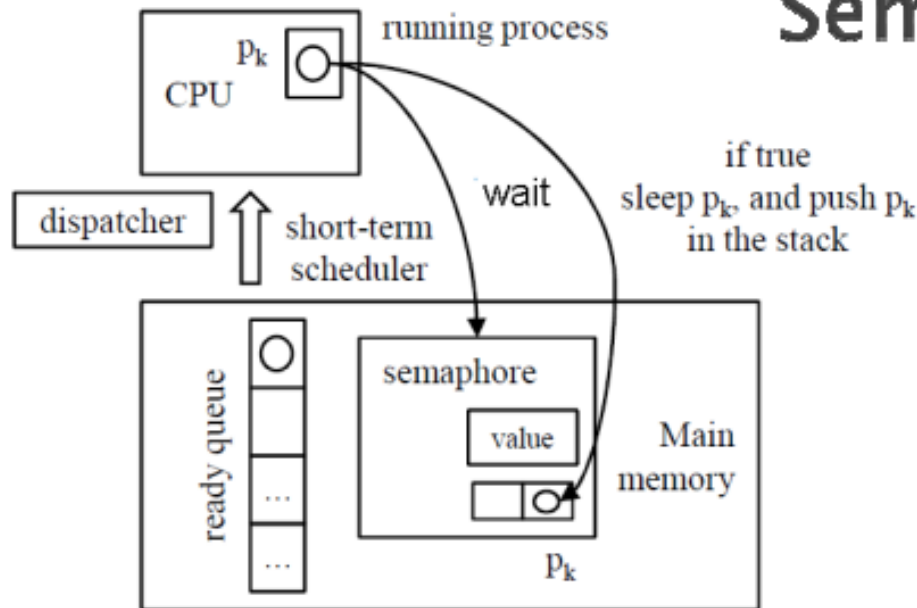
• تبني المسيرات عادة بطريقة الانتظار غير الشاغل

- يتم ذلك بإضافة رتل للعمليات التي نود جعلها في حالة انتظار غير شاغل
- يدير المسير عمليات المزامنة باستخدام الرتل عن طريق التنويم و الايقاظ

- **sleep()** : حيث يتم من خلالها إضافة العملية التي لا تتمكن من الدخول إلى المسير إلى رتل الانتظار و يتم تغيير حالة هذه العملية إلى حالة الانتظار (حالة خاصة من الانتظار blocked)

- **wakeup()** : حيث يتم استدعاؤها من قبل العملية التي تغادر المسير لتقوم بنقل إحدى العمليات الموجودة في الرتل من حالة الانتظار إلى حالة الجاهزية و بالتالي تكون قادرة على متابعة التنفيذ داخل المسير.

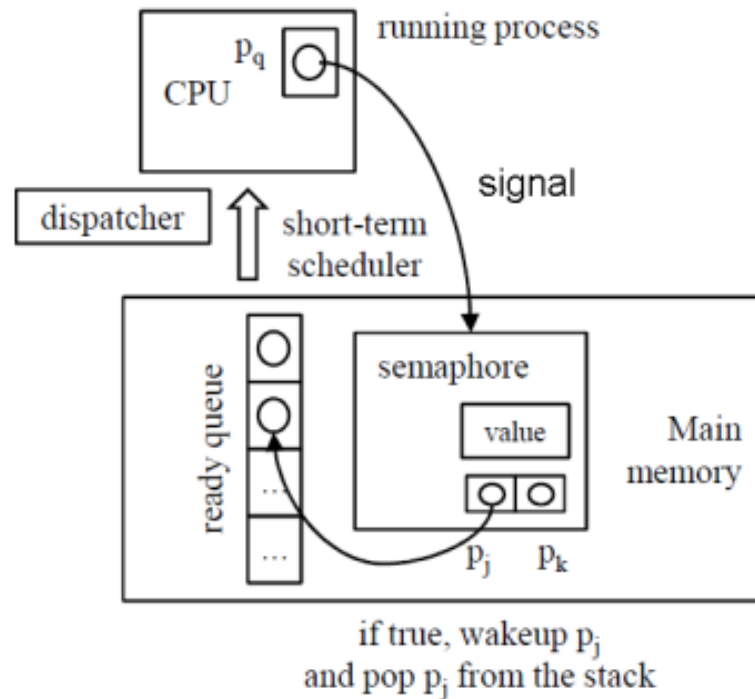
المسيرات - Semaphores



بنية المسيرات الداخلية

- بطريقة الانتظار غير الشاغل

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}
```



المسيرات - Semaphores

بنية المسيرات الداخلية

- بطريقة الانتظار غير الشاغل

```

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}

```

المسيرات - Semaphores

طرق استعمال المسيرات

- يمكن استعمال المسيرات الثنائية (Mutex) للتحكم في تسلسل تنفيذ تعليمات (كتل من التعليمات) تتبع لعمليات مختلفة

```
int pid = fork();  
if (pid > 0)  
    printf("AAAA"); //  
Instruction A  
if (pid == 0)  
    printf("BBBB"); //  
Instruction B
```

باستخدام المسيرات يطلب تعديل النص البرمجي التالي بحيث يتم تنفيذ
التعليمة A قبل التعليمة B ، بافتراض عدم حدوث أخطاء أثناء تنفيذ
ال fork()

طرق استعمال المسيرات

- يمكن استعمال المسيرات الثنائية (Mutex) للتحكم في تسلسل تنفيذ تعليمات (كتل من التعليمات)

تتبع لعمليات مختلفة

```
int pid = fork();  
if (pid > 0)  
    printf("AAAA"); //  
Instruction A  
if (pid == 0)  
    printf("BBBB"); //  
Instruction B
```

باستخدام المسيرات يطلب تعديل النص البرمجي التالي بحيث يتم تنفيذ
التعليمة A قبل التعليمة B ، بافتراض عدم حدوث أخطاء أثناء تنفيذ
ال fork()

```
Symaphore mutex; //initialized to 0  
int pid = fork();  
if (pid >= 0) {  
    printf("AAAA"); // Instruction A  
    signal (mutex);};  
if (pid == 0) {  
    wait (mutex);  
    printf("BBBB"); }; // Instruction B
```



```

Int a=2, b=3, c=4;
Symaphore mutex; //initialized to 0
int pid = fork(); //alpha fork
if (pid > 0) {
    wait (mutex);
    a = a+b +1;    b = c/2;    c = a*b /c;
    printf("a= %d , b=%d , c= %d \n",a,b,c); // Instruction A
}
if (pid == 0) {
    a = b+3*a;    b = b*c;    c = 3*b/a;
    int pid1 = fork(); //beta fork
    if (pid1 == 0) {
        printf("a= %d , b=%d , c= %d \n",a,b,c); // Instruction B
        signal(mutex);
        a=b/3 ;    b=c-a;    c=a/c+b;};
    if (pid1 > 0) {
        a = c-b;    b = 2 - a;    c = c - a - b; };
    }
}

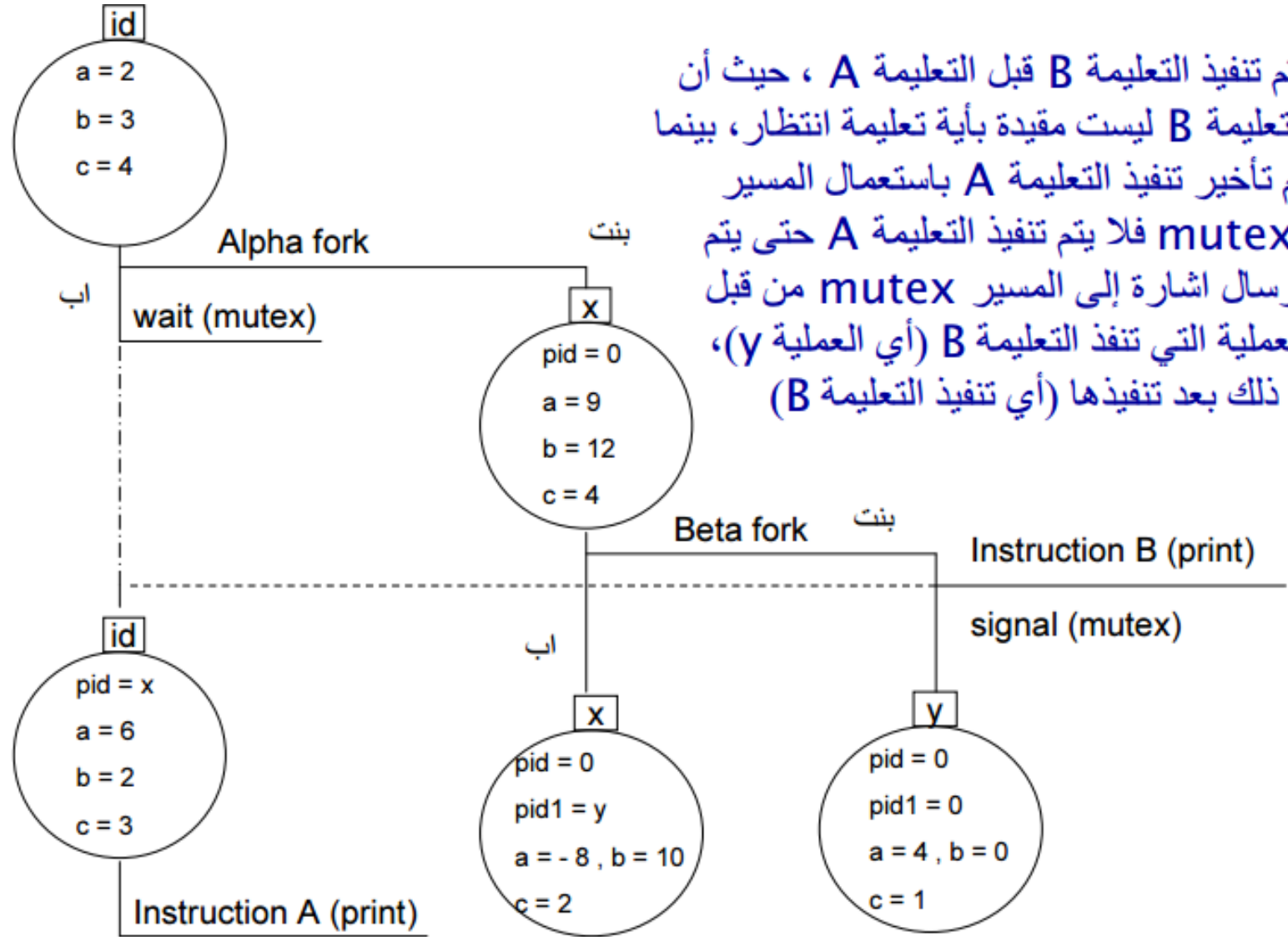
```

بعد امعان النظر في النص البرمجي التالي و تتبع تنفيذه بافتراض عدم حدوث أخطاء أثناء تنفيذ أي من تعليمات الـ fork ، يطلب

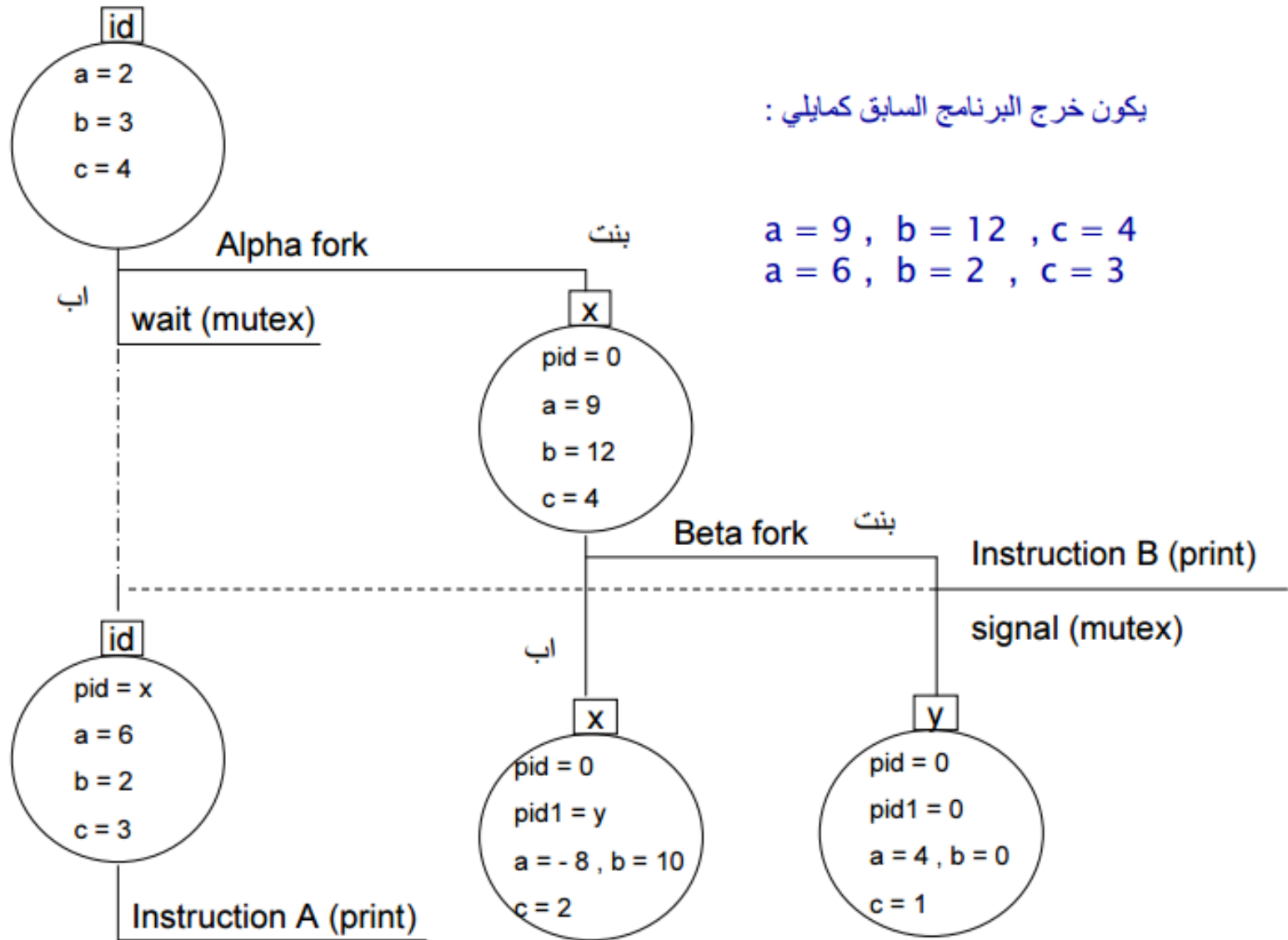
١ - توضيح البنية الشجرية للعمليات الواردة في هذا النص، و توضيح حالة كل من المتحولات بعد كل تعليمة Fork

و توضيح أي من العمليات هي عمليات أم و أي منها هي عمليات ابن.

٢ - توضيح ترتيب تنفيذ العمليات المعلمة بـ A, B مع التعليل و كتابة ما ستقوم هذه التعليمات بطباعته.



يتم تنفيذ العملية B قبل العملية A ، حيث أن العملية B ليست مقيدة بأية عملية انتظار، بينما تم تأخير تنفيذ العملية A باستعمال المسير mutex فلا يتم تنفيذ العملية A حتى يتم إرسال إشارة إلى المسير mutex من قبل العملية التي تنفذ العملية B (أي العملية y)، و ذلك بعد تنفيذها (أي تنفيذ العملية B)



المسيرات - Semaphores

طرق استعمال المسيرات

- يمكن استعمال المسيرات الثنائية (Mutex) لحل مسألة المنطقة الحرجة بين N عملية بحيث يسمح لعملية واحدة فقط بالدخول لمنطقتها الحرجة و تحقيق الاقصاء التبادلي كما يلي:

```
Semaphore mutex; // initialized to 1
do {
    wait (mutex);
    // Critical Section
    signal (mutex);
    // remainder section
} while (TRUE);
```

مسألة المنتج – المستهلك (Producer–Consumer Problem)

تدعى أيضا بمسألة المخزن المحدود (Bounded–Buffer Problem)

- يمكن حل هذه المسألة باستخدام المسيرات كما يلي:

نستخدم المسيرات التالية:

- المسير الثنائي mutex و الذي يأخذ القيمة الابتدائية 1.

- المسير full يأخذ القيمة الابتدائية 0.

- المسير empty و يأخذ القيمة الابتدائية N (سعة المخزن).

مسألة المنتج – المستهلك (Producer-Consumer Problem)

• عملية المنتج (Producer)

```
do {  
    // produce an item in next_produced  
  
    wait (empty);    /* decrement empty count */  
    wait (mutex);    /* enter critical region */  
  
    // add the item to the buffer  
  
    signal (mutex); /* leave critical region */  
    signal (full);  /* increment count of full slots */  
} while (TRUE);
```

مسألة المنتج – المستهلك (Producer-Consumer Problem)

• عملية المستهلك (Consumer)

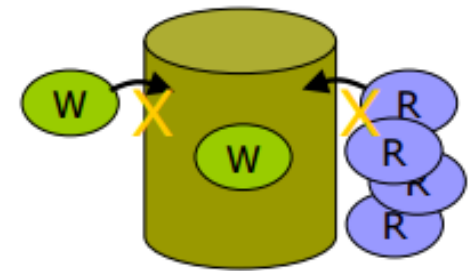
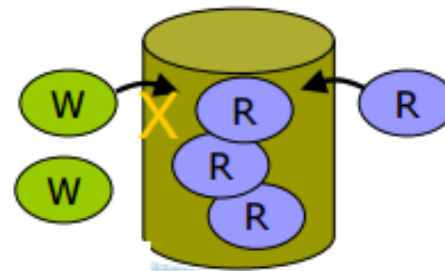
```
do {  
    wait (full);           /* decrement full count */  
    wait (mutex);          /* enter critical region */  
  
    // remove an item from buffer to next_consumed  
  
    signal (mutex);        /* leave critical region */  
    signal (empty);        /* increment count of empty slots */  
  
    // consume the item in next_consumed  
  
} while (TRUE);
```

مسألة القراء و الكتاب (Readers and Writer Problem)

- لتكن لدينا بيانات مشتركة بين مجموعة عمليات بحيث نميز بين نوعين من هذه العمليات:
 - القراء: يقومون فقط بقراءة هذه البيانات دون إجراء أي تعديلات على هذه البيانات
 - الكتاب: يمكن أن يقوموا بقراءة و كتابة البيانات المشتركة
- نود السماح لعدة قراء بأن يقوموا بالقراءة في نفس الوقت بينما يسمح لكاتب واحد فقط بالتعامل مع البيانات المشتركة دون السماح لأي عملية أخرى بالوصول إلى هذه البيانات،
- أي بحيث أن عمليات القراءة لا تقوم بالانتظار إلا إذا كان هناك عملية كتابة ما تقوم بتعديل على البيانات المشتركة.
- و المطلوب استعمال المسيرات في كتابة نص برمجي يسمح بذلك لكل نوع من العمليات (القراء + الكتاب)

مسألة القراء و الكتاب (Readers and Writer Problem)

- لتكن لدينا بيانات مشتركة بين مجموعة عمليات بحيث نميز بين نوعين من هذه العمليات:
- القراء: يقومون فقط بقراءة هذه البيانات دون إجراء أي تعديلات على هذه البيانات
 - الكتاب: يمكن أن يقوموا بقراءة و كتابة البيانات المشتركة
 - نود السماح لعدة قراء بأن يقوموا بالقراءة في نفس الوقت بينما يسمح لكتاب واحد فقط بالتعامل مع البيانات المشتركة دون السماح لأي عملية أخرى بالوصول إلى هذه البيانات،
 - أي بحيث أن عمليات القراءة لا تقوم بالانتظار إلا إذا كان هناك عملية كتابة ما تقوم بتعديل على البيانات المشتركة.
 - و المطلوب استعمال المسيرات في كتابة نص برمجي يسمح بذلك لكل نوع من العمليات (القراء + الكتاب)
 - إما كاتب واحد أو عدة قراء يسمح لهم باستعمال قاعدة البيانات



مسألة القراء و الكتاب (Readers and Writer Problem)

- لحل المسألة نستخدم البيانات المشتركة التالية:
- مسير ثنائي wrt يأخذ القيمة الابتدائية 1
- عدد صحيح readcount يأخذ القيمة الابتدائية 0
- مسير ثنائي mutex يأخذ القيمة الابتدائية 1
- فتكون عملية الكاتب (Writer Process)

```
do {  
    wait (wrt) ;  
  
    // writing is performed  
  
    signal (wrt) ;  
} while (TRUE);
```

مسألة القراء و الكتاب (Readers and Writer Problem)

• و تكون عملية القارئ (Reader Process)

```
do {  
    wait (mutex) ; /* Allow 1 reader in entry */  
    readcount ++ ;  
    if (readcount == 1)  
        wait (wrt) ; /* 1st reader locks writer */  
    signal (mutex)  
  
    // reading is performed  
  
    wait (mutex) ;  
    readcount -- ;  
    if (readcount == 0)  
        signal (wrt) ; /*last reader frees writer */  
    signal (mutex) ;  
} while (TRUE);
```

طرق استعمال المسيرات

- يمكن استعمال المسيرات للتحكم في استعمال عدة موارد K مشتركة تتسابق عليها عدة عمليات N
 - نعرف مسيرا عدادا و نعطيه القيمة الابتدائية k (عدد الموارد المتوفرة)
 - هذا من شأنه أن يسمح لـ k عملية بالدخول إلى مناطقها الحرجة بأن واحد، كل منها يستعمل أحد الـ k مورد المتوفرة
 - عندما تريد أي عملية استعمال أحد الموارد فإنها تنتظر دورها في الحصول عليه باستدعاء `wait ()` مخفضة بذلك عداد المسير
 - عندما تحرر أي عملية موردا انتهت من استعماله فإنها تستدعي `signal ()` لزيادة قيمة عداد المسير
- عندما يكون العداد صفريا (قيمه صفر) فهذا يعني أن جميع الوارد مستعملة، و أن أية عملية تود الحصول على مورد فعليها أن تنتظر حتى يتحرر أحد الموارد لى الأقل (يصبح العداد أكبر من الصفر)

الاستعصاء, المجاعة و عكس الأولوية

- **الاستعصاء (Deadlock):** نقول عن عمليتين أو أكثر أنها في حالة استعصاء إذا كانت كل عملية في حالة انتظار لحدث ما و لا يمكن لهذا الحدث أن يتحقق إلا من قبل عملية أخرى من نفس مجموعة هذه العمليات التي في حالة انتظار.



- **المجاعة (Starvation):** هي الدخول في حالة انتظار لا نهائي، بحيث لا يتم نقل العملية من رتل الانتظار للمسير إلى حالة الجاهزية.
- **عكس الأولوية (Priority Inversion):** هي مشكلة جدولة تمتلك فيها عملية ذات أولوية أدنى قفلا أو مسيرا تكون عملية ذات أولوية أعلى بحاجة للحصول عليه.

مشاكل المسيررات

- يتطلب الاعتماد على المسيررات كأداة للمزامنة حذرا في استخدام العمليتين `wait()` و `signal()` بحيث يتم استدعاء هذه العمليات بالشكل الصحيح.
- هناك العديد من الأخطاء الشائعة في التعامل مع هاتين العمليتين الأمر الذي يتسبب بفشل في آلية المزامنة و بالتالي عدم القدرة على حل مشكلة المناطق الحرجة و تحقيق الاقصاء التبادلي.
- من أكثر هذه الأخطاء شيوعا هي أخطاء الاستخدام التالية:
 - `Wait(mutex) ..... Signal(mutex)`
يتسبب هذا الاستخدام الخاطئ بفشل الاقصاء التبادلي نتيجة لتمكن جميع العمليات من الدخول إلى مناطقها الحرجة.
 - `Wait(mutex) ..... Wait(mutex)`
يتسبب هذا الاستخدام الخاطئ بحدوث استعصاء.
 - حذف العملية `Wait(mutex)` مما يؤدي إلى فشل الاقصاء التبادلي، أو حذف العملية `Signal(mutex)` مما يؤدي إلى استعصاء.

مشكلة عكس الأولوية (Priority Inversions)

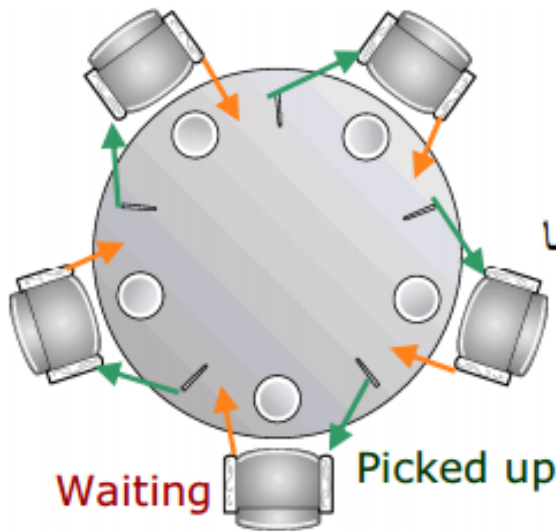
- على فرض أن مسلك ذو أولوية منخفضة قام قفل للـ Mutex مانعاً مسلك ذو أولوية أعلى من الحصول عليه (بالتالي لا يمكن للمسلك ذو الأولوية الأعلى من أن يستخدمه حتى يقوم المسلك الآخر بفك القفل عنه)، فإذا أتى مسلك ذو أولوية متوسطة (بين المسلكين) وهو يريد أن ينفذ على منطقة ليست ضمن المنطقة الحرجة (وبالتالي يطلب جدولة من قبل المعالج ليدخل ضمن التنفيذ) فإذا كان هذا المسلك يحتوي على حلقة لانهائية فعندها المسلك ذو الأولوية العليا يجب أن ينتظر بشكل كبير حتى يحرر المسلك ذو الأولوية المنخفضة للقفل وهذا ما يسمى بعكس الأولوية (أي أن الأولوية هنا لم يعد لها أهمية).

سقف الأولوية (Priority Ceiling)

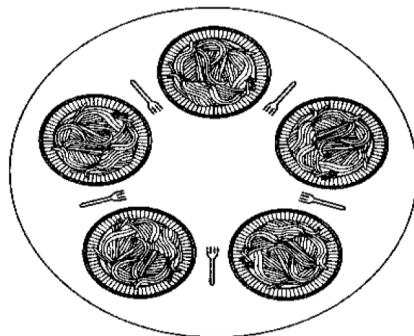
- تعد حل للمشكلة السابقة حيث أنه عندما يقوم أحد المسالك بقفل للـ Mutex عندها سوف يتم رفع الأولوية له إلى أعلى من أعلى مسلك آخر بين المسلك ذو الأولوية المنخفضة و المسلك ذو الأولوية المرتفعة وبالتالي هنا لم يعد يوجد مسلك ذو أولوية متوسطة.

مسألة غداء الفلاسفة (Dining-Philosopher Problem)

لدينا خمسة فلاسفة يقضون حياتهم بين الأكل و التفكير و لكنهم لا يستطيعون التفكير أثناء الأكل، كما أنهم لا يستطيعون الأكل أثناء التفكير، فإذا علمت أن



- هؤلاء الفلاسفة يجلسون على طاولة مستديرة يوجد في منتصفها صحن من الرز
- بجوار كل فيلسوف يوجد عصا واحدة فقط للأكل عن يمينه وعصا واحدة فقط للأكل عن يساره
- الفيلسوف يستطيع استخدام العصا التي عن يمينه مباشرة أو عن يساره مباشرة فقط
- من المستحيل تناول الرز بعصا واحدة أي أن الفيلسوف يحتاج لعصاتين ليتمكن من الأكل
- عندما ينتهي من الأكل يضع العصي على الطاولة في أمكنتها سامحاً بذلك لفيلسوف آخر مجاور أن يلتقط إحداها (المجاورة لهذا الأخير) ثم يعود للتفكير من جديد
- الفلاسفة لا تتحدث مع بعضها أثناء الجلوس



مسألة غداء الفلاسفة (Dining-Philosopher Problem)

• نستخدم مصفوفة من المسمّرات الثنائية chopstick[5] تأخذ جميعها القيمة الابتدائية 1

• و يكون الفيلسوف رقم (i) while (true) {

wait(chopStick [i]); // get left chopstick

wait(chopStick [(i + 1) % 5]); // get right chopstick

// eat for a while

signal(chopStick [i]); //return left chopstick

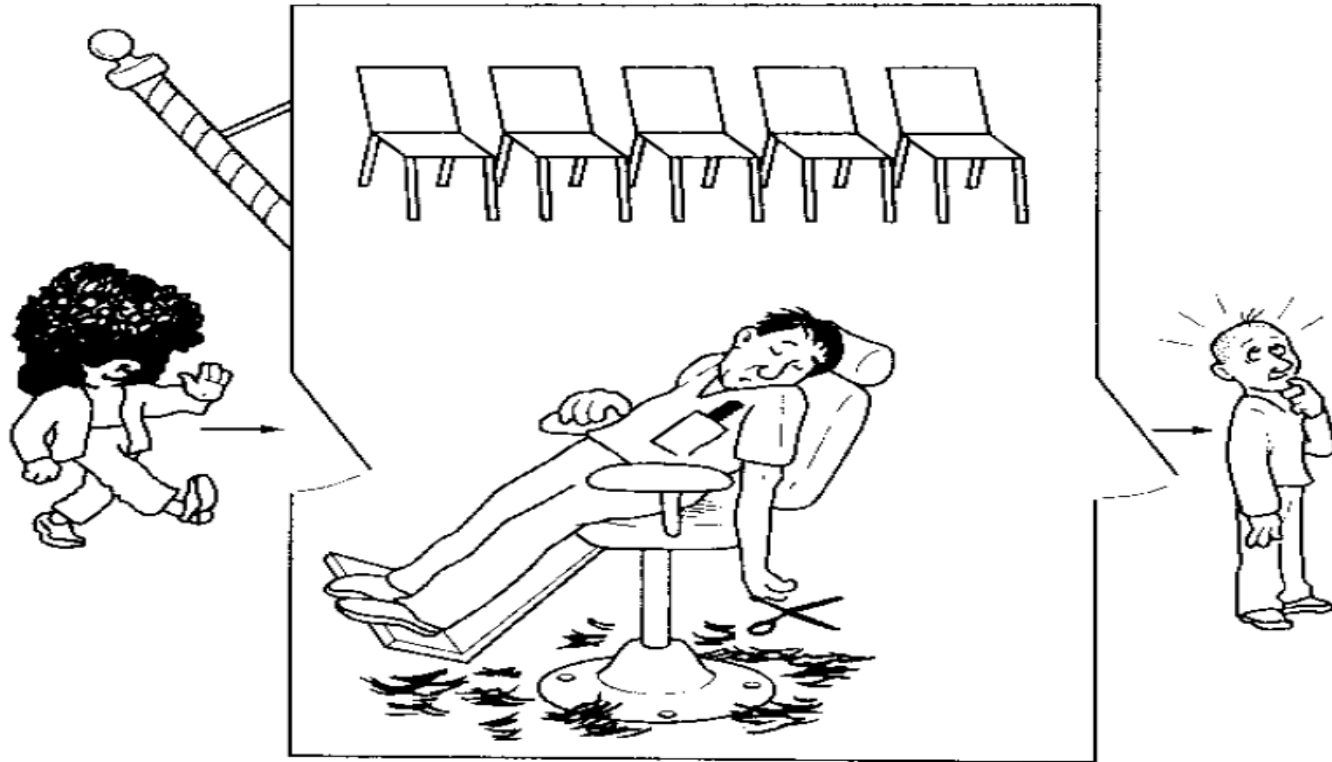
signal (chopStick [(i + 1) % 5]); // return right chopstick

// think for awhile }

حل مشكلة الحلاق النائم (Sleeping Barber):

إذا افترضنا أن لدينا صالون حلاقة به حلاق واحد وكُرسي حلاقة واحد وعدد من الكراسي للإنتظار. إذا لم يكن هنالك زبائن سيجلس الحلاق في كرسي الحلاقة وينام، وعندما يصل زبون فسيقوم بإيقاظ الحلاق. إذا جاء زبون آخر وكان الحلاق يحلق للزبون الأول فسيجلس الزبون الثاني على أحد كراسي الإنتظار. وكلما يصل زبون سيجلس في كرسي إنتظار إلى أن تمتلي كراسي الإنتظار. إذا جاء زبون ووجد كراسي الإنتظار مشغولة فعليه مغادرة الصالون.

الحل



حل مشكلة الحلاق النائم (Sleeping Barber):

لحل هذه المشكلة نستخدم ثلاث سيمافورات:

- سيمافور للزبائن المنتظرين
- سيمافور للحلاق (لنعرف هل هو نائم (عاطل) أم يعمل)
- سيمافور لتحقيق والتأكد من المنع المتبادل (mutual exclusion): بحيث لا نسمح لشخصين بالحلاقة في وقت واحد.

كل ما يحضر زبون سيحاول الحصول على كرسي الحلاقة (mutex) وسيظل كذلك حتى ينجح. على الزبون الذي يحضر للصالون أن يحسب عدد الزبائن المنتظرين فإذا كان أقل من عدد الكراسي فسيجلس وإلا فسيغادر (يحاول الحصول على كرسي سواء في غرفة الإنتظار أو كرسي الحلاق نفسه).

حل مشكلة الحلاق النائم (Sleeping Barber):

إذا وجد الزبون كرسي سيجلس وينقص عدد الكراسي الفارغة بواحد (critical section).

يقوم الزبون بإرسال إشارة إلى الحلاق لإيقاظه، وسيحرر mutex للسماح للزبائن (أو الحلاق) بالمقدرة على الحصول عليه. إذا كان الحلاق مشغول فعلى الزبائن الانتظار. سيجل الحلاق في إنتظار دائم، يتم ايقاظه بأي زبون من المنتظرين، وعندما يستيقظ سيرسل إشارات للزبائن المنتظرين بواسطة السيمافور للسماح لهم بالحلاقة ، واحد كل مرة. هذه المشكلة تحتوي على حلاق واحد لذلك تسمى أحيانا (*single sleeping barber problem*). فيما يلي شفرات مبدئية (pseudo-code)، تتضمن التزامن بين الحلاق والزبائن خالية من الاختناق، ولكنها قد تقود إلى حرمان الزبون.

نجد V و P هما دالتين توفرهما السيمافورات.

Semaphore Customers = 0

Semaphore Barber = 0

Semaphore accessSeats (mutex) = 1

حل مشكلة الحلاق النائم (Sleeping Barber):

```
#define CHAIRS 5
typedef int semaphore;
semaphore customers = 0;
semaphore barbers = 0;
semaphore mutex = 1;
int waiting = 0;

/* number of chairs for waiting customers */
/* number of waiting customers */
/* number of barbers waiting for customers */
/* for mutual exclusion */
/* customers are waiting not being haircut */

void Barber(void)
{
    while (TRUE)
    {
        down(customers); /* go to sleep if number of customers is 0 */
        down(mutex); /* acquire access to 'waiting' */
        waiting = waiting - 1; /* decrement count of waiting customers */
        up(barbers); /* one barber is now ready to cut hair */
        up(mutex); /* release 'waiting' */
        cut_hair(); /* cut hair, non-CS */
    }
}

void customer(void)
{
    down(mutex); /* enter CS */
    if (waiting < CHAIRS)
    {
        waiting = waiting + 1; /* increment count of waiting customers */
        up(customers); /* wake up barber if necessary */
        up(mutex); /* release access to 'waiting' */
        down(barbers); /* wait if no free barbers */
        get_haircut(); /* non-CS */
    }
    else
    {
        up(mutex); /* shop is full, do not wait */
    }
}
```

المراقبات - Monitors

- لجعل كتابة البرامج الصحيحة أسهل، تم تقديم أداة عالية المستوى لتحقيق المزامنة تسمى المراقب (Monitor).
- المراقب هو عبارة عن مجموعة إجراءات و متحولات و بنى معطيات مجمعة بنوع خاص من الوحدات النمطية (Classes)، بحيث تستطيع العمليات استدعاء الإجراءات الموجودة في المراقب لكنها لا تستطيع الوصول مباشرة إلى البيانات الداخلية في المراقب انطلاقاً من إجراءات لا تنتمي إلى المراقب.

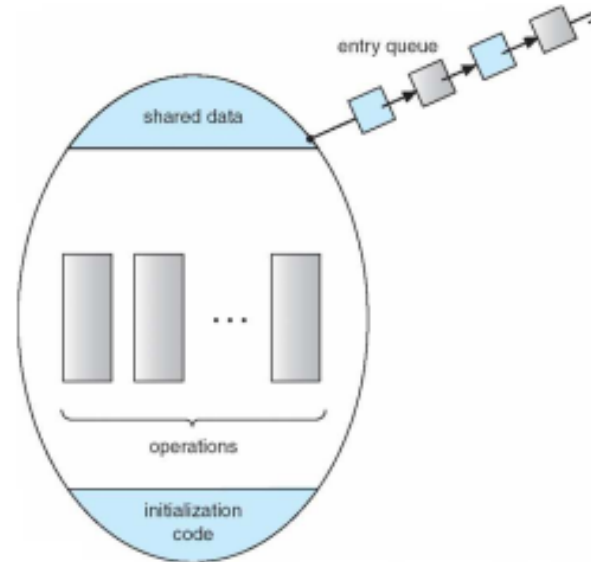
```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { .... }
    ...

    procedure Pn (...) {.....}

    Initialization code ( ....) { ... }
    ...
}
```

المراقبات - Monitors

- للمراقبات خاصية هامة تجعلها مفيدة لتحقيق الاقصاء التبادلي، و هي أن عملية واحدة فقط تستطيع أن تكون نشطة في المراقب في أي لحظة زمنية، بحيث يتولى المترجم (Compiler) تحقيق ذلك و ليس المبرمج.
- عندما تستدعي عملية إجراء مراقب، فإنه سيتم تفحص ما إذا كانت أي عملية أخرى نشطة داخل المراقب بحيث يجب على العملية المستدعية الانتظار ريثما تخرج العملية الأخرى من المراقب و عندها يسمح للعملية المستدعية بالدخول.

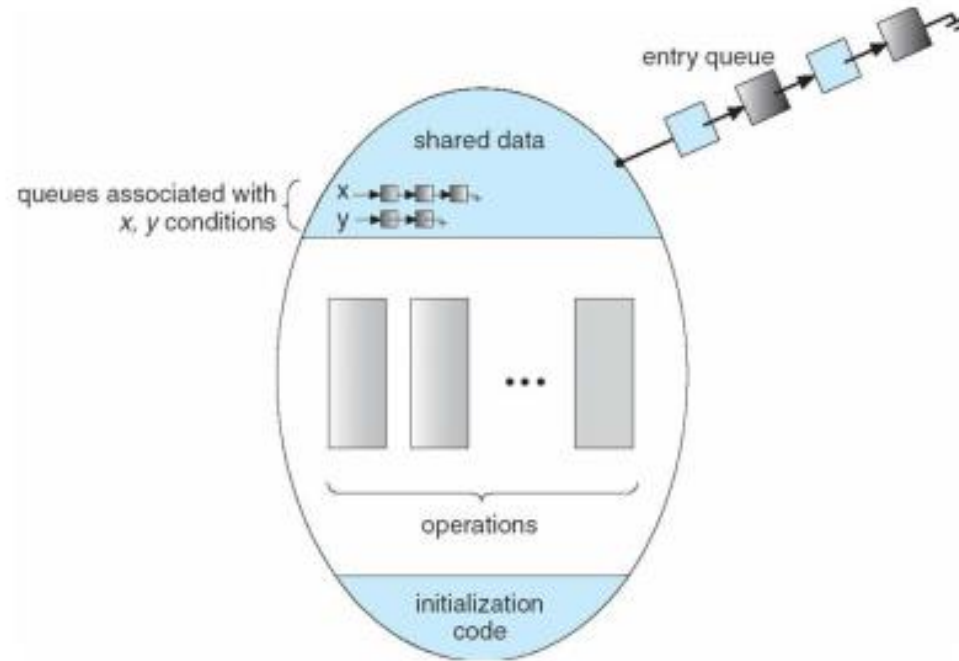


المراقبات - Monitors

- بتحويل جميع المناطق الحرجة إلى إجراءات في مراقب فلن تستطيع أي عمليتين الدخول إلى منطقتيهما الحرجتين في نفس الوقت مما يحقق الاقصاء التبادلي.
- على الرغم من أن المراقبات تقدم طريقة سهلة لتحقيق الاقصاء التبادلي، فإن ذلك ليس كافياً، لأننا نحتاج أيضاً إلى إيقاف العمليات عندما لا تستطيع متابعة التنفيذ، كما في مسألة المنتج - مستهلك مثلاً.
- يكمن الحل في تقديم متحولات شرطية (Condition Variables) مع العمليتين wait() و signal() على هذه المتحولات.
- عندما يكتشف إجراء ضمن المراقب أنه لا يستطيع المتابعة فإنه ينفذ wait() على أحد متحولات الشرط، مما يؤدي إلى إيقاف العملية المستدعية كما يسمح لعملية أخرى بالدخول إلى المراقب.
- تستطيع عملية ما إيقاف عملية ثانية بتنفيذ العملية signal() على متحول الشرط الذي تنتظره تلك العملية.

المراقبات - Monitors

- متحولات الشرط ليس لها عدادات لأنها لا تراكم الإشارات من أجل الاستخدام اللاحق كما تفعل المسيرات، و بالتالي إذا تم تأشير (Signal()) متحول شرط دون وجود عملية تنتظر عنده فإن الإشارة ستضيع للأبد.



حل مسألة غداء الفلاسفة باستخدام المراقبات

- يجب على كل فيلسوف أن يستدعي العمليتين `pickup()` و `putdown()` وفق التسلسل التالي:

```
DiningPhilosphers.pickup (i);
```

EAT

```
DiningPhilosophers.putdown (i);
```

حل مسألة غداء الفلاسفة باستخدام المراقبات

monitor DP

```
{
    enum { THINKING; HUNGRY, EATING) state [5] ;
    condition self [5];

    initialization_code() {
        for (int i = 0; i < 5; i++)
            state[i] = THINKING;
    }
    void test (int i) {
        if ( (state[(i + 4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i + 1) % 5] != EATING) ) {
            state[i] = EATING ;
            self[i].signal () ;
        }
    }
}
```

حل مسألة غداء الفلاسفة باستخدام المراقبات

```
void pickup (int i) {  
    state[i] = HUNGRY;  
    test(i);  
    if (state[i] != EATING) self [i].wait;  
}
```

```
void putdown (int i) {  
    state[i] = THINKING;  
    // test left and right neighbors  
    test((i + 4) % 5);  
    test((i + 1) % 5);  
}}
```

[illegible]